

UNIT II – Linear Data Structures, searching and sorting

2.1 Overview of Arrays

Learning Objectives

After studying this chapter, students will be able to:

- Define an array and explain its importance.
 - Understand the characteristics of arrays.
 - Explain how arrays are stored in memory.
 - Perform basic array operations.
 - Represent one-dimensional and multidimensional arrays.
 - Understand sparse matrices and their applications.
 - Implement arrays using Python.
 - Apply arrays to solve real-world problems.
-

2.1.1 Introduction to Arrays

Imagine a teacher wants to store the marks of **60 students**.

One approach is to create 60 separate variables:

mark1 = 85

mark2 = 92

mark3 = 76

...

mark60 = 88

This approach is difficult because:

- A large number of variables must be created.
- Searching and updating values becomes cumbersome.
- Programs become lengthy and difficult to maintain.

Instead, all marks can be stored in a single collection:

```
marks = [85, 92, 76, 90, 88]
```

This collection is called an **array**.

An array allows us to store multiple values of the same data type using a single variable name, with each value identified by an index.

Definition

An **array** is a **linear data structure** that stores a fixed number of elements of the **same data type** in **contiguous memory locations**.

Each element is accessed using an **index**, which represents its position in the array.

Formal Definition

An array is a finite, ordered collection of homogeneous elements stored in contiguous memory locations and accessed using an index.

Why Do We Need Arrays?

Without arrays:

- Hundreds of variables are required.
- Processing similar data becomes difficult.
- Searching and sorting operations are inefficient.
- Code becomes difficult to understand and maintain.

Arrays solve these problems by storing related data together.

Real-Life Example 1: Library Book Rack

Imagine a library with books placed on a shelf.

Book 1 Book 2 Book 3 Book 4 Book 5

Each book occupies a specific position.

To retrieve the third book, the librarian simply counts to the third position.

Similarly, array elements are stored sequentially, and each element has a unique index.

Real-Life Example 2: Apartment Building

Consider an apartment building.

Flat 101

Flat 102

Flat 103

Flat 104

Flat 105

Each flat has a unique number.

If someone wants to visit Flat 103, they directly locate that flat.

Similarly, an array element is accessed directly using its index.

Real-Life Example 3: Student Roll Numbers

Roll No. 1

Roll No. 2

Roll No. 3

Roll No. 4

Roll No. 5

Each student has a unique roll number.

Similarly, each array element has a unique index.

Characteristics of Arrays

1. Homogeneous Data

All elements in an array should represent the same type of data.

Example:

marks = [78, 82, 91, 65]

All values are integers.

2. Contiguous Memory Allocation

Array elements are stored in adjacent memory locations.

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| 78 | 82 | 91 | 65 | 88 | 72 | 95 | 81 |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

Since memory is continuous, accessing any element is very fast.

3. Fixed Size

In traditional arrays, the size is decided during creation.

Example:

Size = 5

Only five elements can be stored.

Note: Python lists are dynamic and can grow or shrink, but the concept of arrays in Data Structures assumes a fixed size.

4. Index-Based Access

Each element is accessed using an index.

Index

0 1 2 3 4

↓

10 20 30 40 50

Example:

```
numbers = [10, 20, 30, 40, 50]
```

```
print(numbers[2])
```

Output:

30

5. Direct Access

Arrays provide direct or random access to elements.

Unlike linked lists, there is no need to traverse previous elements.

Access Time:

$O(1)$

Structure of an Array

Index

0 1 2 3 4

+-----+-----+-----+-----+-----+

| 15 | 25 | 35 | 45 | 55 |

+-----+-----+-----+-----+-----+

Components of an Array

Every array consists of:

- Array Name
- Index
- Elements
- Size

Example:

```
marks = [85, 90, 78, 92]
```

Component	Value
Array Name	marks
Size	4
First Index	0
Last Index	3
First Element	85
Last Element	92

Declaration of an Array

General Syntax

```
ArrayName[size]
```

Example:

```
marks[5]
```

Python Representation

Python does not have a built-in fixed-size array like C.

Instead, it commonly uses **lists** to represent arrays.

Example:

```
numbers = [10, 20, 30, 40, 50]
```

Accessing Elements

```
numbers = [12, 24, 36, 48]
```

```
print(numbers[0])
```

```
print(numbers[2])
```

Output

12

36

Traversing an Array

Traversing means visiting every element exactly once.

Example:

```
numbers = [10, 20, 30, 40, 50]
```

```
for number in numbers:
```

```
    print(number)
```

Output

10

20

30

40

Advantages of Arrays

- Easy to understand and implement.
 - Direct access using index.
 - Efficient memory utilization due to contiguous storage.
 - Suitable for searching and sorting algorithms.
 - Fast traversal of elements.
-

Limitations of Arrays

- Fixed size in traditional implementations.
 - Insertion and deletion are costly because elements must be shifted.
 - Can store only homogeneous data (conceptually).
 - Memory may be wasted if the array is underutilized.
-

Applications of Arrays

Arrays are widely used in computer science and engineering.

Education

- Student marks
- Attendance records
- Roll numbers

Banking

- Customer accounts
- Daily transactions

Hospitals

- Patient records
- Bed allocation

Shopping Websites

- Product catalogues
- Shopping carts

Games

- Scoreboards
- Game maps
- Player positions

Image Processing

Digital images are stored as two-dimensional arrays of pixels.

Scientific Computing

Matrices, vectors, and numerical data are represented using arrays.

Python Program: Finding Maximum Element

```
numbers = [15, 28, 9, 42, 31]
```

```
maximum = numbers[0]
```

```
for num in numbers:
```

```
    if num > maximum:
```

```
        maximum = num
```

```
print("Maximum element:", maximum)
```

Output

Maximum element: 42

Time Complexity

$O(n)$

Summary

- An array is a linear data structure.
 - It stores homogeneous elements in contiguous memory.
 - Elements are accessed using an index.
 - Arrays provide constant-time random access.
 - They are widely used in searching, sorting, matrices, image processing, databases, and scientific applications.
 - Although Python uses lists, the array concepts remain fundamental to understanding data structures.
-

Key Points

- Array = Collection of similar elements.
- Memory is contiguous.
- Index starts from **0**.
- Direct access complexity is **$O(1)$** .
- Traversal complexity is **$O(n)$** .
- Traditional arrays have a fixed size.

2.2 Array as an Abstract Data Type (ADT)

(As per SPPU 2024 Pattern – Unit II)

Learning Objectives

After studying this section, students will be able to:

- Understand the concept of an Abstract Data Type (ADT).

- Explain Array as an ADT.
 - Differentiate between an Array and an Array ADT.
 - Understand all operations supported by the Array ADT.
 - Implement Array ADT operations using Python.
 - Analyze the time complexity of array operations.
-

2.2.1 Introduction to Abstract Data Type (ADT)

Before understanding **Array ADT**, let us first understand what an **Abstract Data Type (ADT)** is.

An **Abstract Data Type (ADT)** is a logical or mathematical model of a data structure. It specifies **what operations can be performed** on the data but **does not specify how these operations are implemented**.

In simple words,

ADT describes the behavior of a data structure, not its implementation.

Real-Life Example: Television Remote

Consider a TV remote.

You know that pressing buttons performs operations such as:

- Power ON/OFF
- Volume Up
- Volume Down
- Channel Change

However, you do not need to know the internal electronic circuits that perform these operations.

Similarly, an ADT tells the user **what operations are available**, but hides **how they are implemented**.

Diagram: Concept of ADT

USER

|



Abstract Data Type

(Operations Definition)

|

Insert()

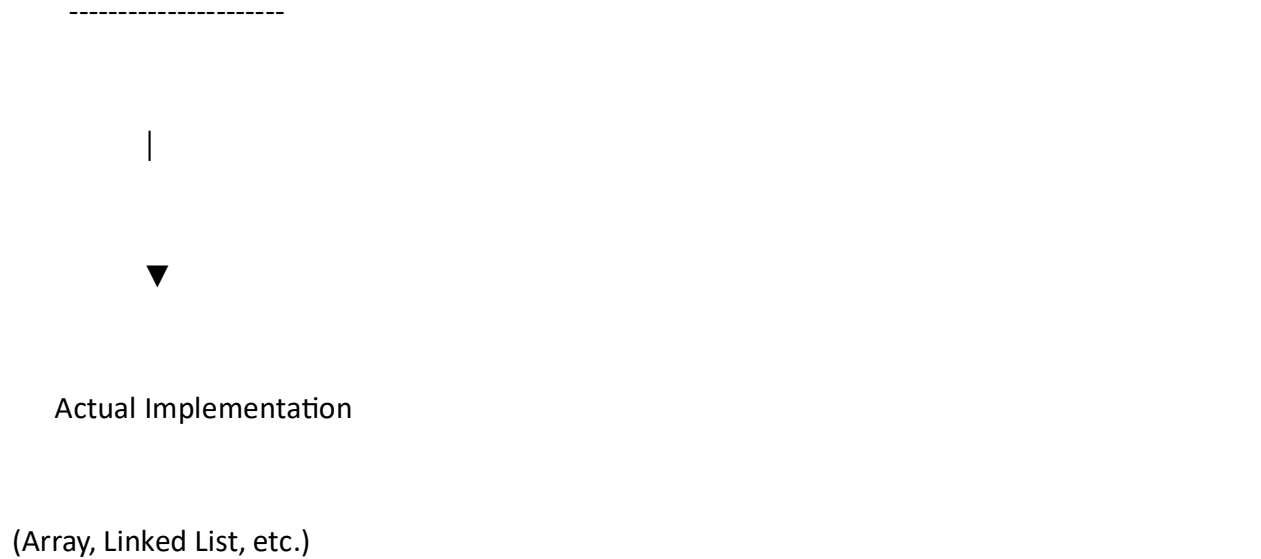
Delete()

Search()

Update()

Traverse()

Sort()



Definition of Array ADT

An **Array ADT** is a logical data type that stores a collection of homogeneous elements and provides predefined operations such as insertion, deletion, searching, updating, and traversal.

The Array ADT specifies **what operations** are possible without specifying **how the array is stored in memory**.

Array ADT vs Array Implementation

Many students confuse these two terms.

Array ADT	Array Implementation
Logical view	Physical implementation
Defines operations	Defines memory representation
Independent of programming language	Depends on language
Describes behavior	Describes implementation
User focuses on functionality	Programmer focuses on coding

Illustration

Array ADT

"Store Student Marks"

|

Insert

Delete

Search

Update

Traverse

|



Can be Implemented Using

Python List

C Array

Java Array

NumPy Array

Characteristics of Array ADT

An Array ADT has the following characteristics:

1. Homogeneous Elements

All elements stored in an array are of the same data type.

Example:

```
marks = [85, 90, 76, 88]
```

All elements are integers.

2. Ordered Collection

Elements are arranged in a fixed sequence.

Example

10 → 20 → 30 → 40 → 50

3. Index-Based Access

Every element is identified using an index.

Index

0 1 2 3



15 25 35 45

4. Fixed Size (Traditional Arrays)

Traditional arrays have a predefined size.

Example

Array Size = 5

Only five elements can be stored.

5. Random Access

Elements can be accessed directly.

Complexity

$O(1)$

Operations Supported by Array ADT

The Array ADT supports several basic operations.

Array ADT

|

Traverse

Insert

Delete

Search

Update

Sort

Merge

Reverse

1. Traverse Operation

Definition

Traversal means visiting each element of the array exactly once.

Real-Life Example

Teacher checking attendance of every student.

Roll 1

↓

Roll 2



Roll 3



...



Roll n

Python Program

```
numbers = [10,20,30,40,50]
```

```
for item in numbers:
```

```
    print(item)
```

Output

10

20

30

40

50

Complexity

$O(n)$

2. Insert Operation

Definition

Insertion means adding a new element into an array.

Types

- Insert at Beginning
 - Insert at Middle
 - Insert at End
-

Example

Before insertion

10 20 40 50

Insert 30

After insertion

10 20 30 40 50

Python Program

```
numbers = [10,20,40,50]
```

```
numbers.insert(2,30)
```

```
print(numbers)
```

Output

```
[10,20,30,40,50]
```

Complexity

$O(n)$

3. Delete Operation

Definition

Deletion removes an existing element from the array.

Example

Before

10 20 30 40 50

Delete

30

After

10 20 40 50

Python Program

```
numbers=[10,20,30,40,50]
```

```
numbers.remove(30)
```

```
print(numbers)
```

Output

```
[10,20,40,50]
```

Complexity

$O(n)$

4. Search Operation

Searching means locating an element.

Example

Search 40

↓

Found at Index 3

Python

```
numbers=[10,20,30,40]
```

```
print(numbers.index(40))
```

Output

3

Complexity

Linear Search

$O(n)$

Binary Search

$O(\log n)$

(Will be discussed later in this unit.)

5. Update Operation

Updating means replacing an existing value.

Example

Before

10 20 30 40

Update

30 → 35

After

10 20 35 40

Python

```
numbers=[10,20,30,40]
```

```
numbers[2]=35
```

```
print(numbers)
```

Output

```
[10,20,35,40]
```

Complexity

O(1)

6. Sort Operation

Sorting arranges elements in ascending or descending order.

Before

45 10 25 18

After

10 18 25 45

Python

```
numbers=[45,10,25,18]
```

```
numbers.sort()
```

```
print(numbers)
```

Output

```
[10,18,25,45]
```

7. Merge Operation

Combining two arrays into one.

A

10 20 30

+

B

40 50

↓

Merged

10 20 30 40 50

Python

```
A=[10,20,30]
```

```
B=[40,50]
```

`C=A+B`

`print(C)`

Output

`[10,20,30,40,50]`

8. Reverse Operation

Before

`10 20 30 40`

After

`40 30 20 10`

Python

`numbers=[10,20,30,40]`

`numbers.reverse()`

`print(numbers)`

Output

`[40,30,20,10]`

Time Complexity of Array ADT Operations

Operation	Time Complexity
Access	$O(1)$
Update	$O(1)$
Traverse	$O(n)$

Search (Linear)	$O(n)$
Search (Binary)	$O(\log n)$
Insert	$O(n)$
Delete	$O(n)$
Sort	Depends on algorithm (e.g., Bubble: $O(n^2)$, Merge: $O(n \log n)$)
Merge	$O(n + m)$
Reverse	$O(n)$

Real-Life Applications of Array ADT

Student Database

- Insert a new student.
- Delete a student who leaves.
- Update student marks.
- Search by roll number.
- Sort students based on marks.

Library Management System

- Add a new book.
- Remove a lost book.
- Search for a title.
- Update book details.
- Sort books alphabetically.

E-commerce Website

- Add products.
- Remove unavailable products.
- Search for products.
- Update prices.

- Sort by price or rating.
-

Advantages of Array ADT

- Easy to understand.
 - Provides a clear interface.
 - Supports random access.
 - Suitable for implementing many algorithms.
 - Language independent concept.
-

Limitations

- Fixed size in traditional arrays.
 - Insertion and deletion require shifting elements.
 - Memory wastage may occur.
 - Stores homogeneous data only (conceptually).
-

Difference Between ADT and Data Structure

Abstract Data Type (ADT)	Data Structure
Defines what operations are allowed	Defines how data is stored
Logical model	Physical implementation
Independent of language	Language dependent
Hides implementation	Shows implementation
Example: Array ADT	Example: Python List, C Array

Summary

- An ADT specifies the operations that can be performed on data without defining how those operations are implemented.

- Array ADT represents an ordered collection of homogeneous elements.
- Common operations include traversal, insertion, deletion, searching, updating, sorting, merging, and reversing.
- Array ADT separates the logical behavior of arrays from their physical implementation.
- Understanding Array ADT helps in designing efficient algorithms and choosing appropriate implementations.

2.3 Operations on Arrays

Learning Objectives

After studying this section, students will be able to:

- Explain the basic operations performed on arrays.
 - Write algorithms and Python programs for array operations.
 - Analyze the time complexity of each operation.
 - Apply array operations in real-world applications.
-

2.3.1 Introduction

An array is useful only when we can perform different operations on it.

The most common array operations are:

ARRAY OPERATIONS

Traverse Insert Delete Search Update Sort Merge Reverse

These operations are widely used in database systems, banking applications, student management systems, hospital management software, and many other real-world applications.

2.3.2 Traversing an Array

Definition

Traversal means visiting every element of an array exactly once to process or display its value.

Real-Life Example

A teacher checks the attendance of every student in a classroom one by one.

Student 1



Student 2



Student 3



...



Student n

Every student is visited once.

This is traversal.

Algorithm

Algorithm Traverse(A,n)

1. Start

2. For i = 0 to n-1

Display A[i]

3. Stop

Python Program

```
numbers = [12, 25, 40, 18, 55]
```

```
for element in numbers:
```

```
    print(element)
```

Output

12

25

40

18

55

Complexity

Best	Average	Worst
O(n)	O(n)	O(n)

2.3.3 Insertion Operation

Definition

Insertion means adding a new element into an array at a specified position.

Types of Insertion

(A) Insertion at Beginning

Example

Before

Index

0 1 2 3

10 20 30 40

Insert

5

After

5 10 20 30 40

All elements shift one position to the right.

Python Program

```
numbers = [10,20,30,40]
```

```
numbers.insert(0,5)
```

```
print(numbers)
```

Output

```
[5,10,20,30,40]
```

(B) Insertion in Middle

Before

10 20 40 50

Insert

30

After

10 20 30 40 50

Python

```
numbers=[10,20,40,50]
```

```
numbers.insert(2,30)
```

```
print(numbers)
```

(C) Insertion at End

```
numbers=[10,20,30]
```

```
numbers.append(40)
```

```
print(numbers)
```

Output

```
[10,20,30,40]
```

Algorithm

Algorithm Insert(A,n,pos,item)

1. Shift all elements from last position to pos

2. Insert item

3. Increase size

4. Stop

Complexity

Best Average Worst

$O(1)$ * $O(n)$ $O(n)$

*Appending at the end of a dynamic array (such as a Python list) is often $O(1)$ on average.

Real-Life Example

Suppose a new student takes admission.

Current roll numbers

1

2

3

4

New student joins at Roll No. 2.

Students after Roll No. 2 shift one position.

2.3.4 Deletion Operation

Definition

Deletion means removing an existing element from an array.

Example

Before

10 20 30 40 50

Delete

30

After

10 20 40 50

Remaining elements shift left.

Algorithm

Algorithm Delete(A,n,pos)

1. Shift all elements after pos one position left
 2. Reduce array size
 3. Stop
-

Python Program

```
numbers=[10,20,30,40,50]
```

```
numbers.remove(30)
```

```
print(numbers)
```

Output

```
[10,20,40,50]
```

Complexity

Best Average Worst

$O(1)$ $O(n)$ $O(n)$

*Deleting the last element of a dynamic array can be $O(1)$.

Real-Life Example

Suppose one employee resigns.

All employees after that employee move one position upward in the employee list.

2.3.5 Searching Operation

Definition

Searching is the process of locating an element in an array.

Example

Array

10 20 30 40 50

Search

40



Found at Index 3

Python Program

```
numbers=[10,20,30,40,50]
```

```
key=40
```

```
if key in numbers:
```

```
    print("Found")
```

```
else:
```

```
    print("Not Found")
```

Output

Found

Complexity

Searching Method Complexity

Linear Search	$O(n)$
Binary Search	$O(\log n)$

Note: Linear Search, Binary Search, Fibonacci Search, and Indexed Sequential Search are covered in detail later in this unit.

Real-Life Example

Searching a contact in a mobile phone.

You enter the name and the phone finds the contact.

2.3.6 Updating Operation

Definition

Updating means changing the value of an existing element.

Example

Before

10 20 30 40

Update

30 → 35

After

10 20 35 40

Python Program

```
numbers=[10,20,30,40]
```

```
numbers[2]=35
```

```
print(numbers)
```

Output

```
[10,20,35,40]
```

Complexity

O(1)

Real-Life Example

Updating a student's marks after revaluation.

2.3.7 Merging Two Arrays

Definition

Merging means combining two arrays into a single array.

Example

Array A

10 20 30

Array B

40 50 60

↓

Merged

10 20 30 40 50 60

Python Program

```
A=[10,20,30]
```

```
B=[40,50,60]
```

```
C=A+B
```

```
print(C)
```

Output

```
[10,20,30,40,50,60]
```

Complexity

$O(n+m)$

where **n** and **m** are the sizes of the two arrays.

Real-Life Example

Combining sales records from two different branches into one report.

2.3.8 Reversing an Array

Definition

Reversing means arranging the elements in the opposite order.

Before

```
10 20 30 40
```

After

```
40 30 20 10
```

Python Program

```
numbers=[10,20,30,40]
```

```
numbers.reverse()
```

```
print(numbers)
```

Output

```
[40,30,20,10]
```

Complexity

$O(n)$

Real-Life Example

Reversing the order of songs in a playlist.

2.3.9 Complete Comparison of Array Operations

Operation	Description	Time Complexity
Traversal	Visit every element	$O(n)$
Insertion (Beginning/Middle)	Add an element with shifting	$O(n)$
Insertion (End)	Add an element at the end	$O(1)$ average (dynamic arrays)
Deletion (Beginning/Middle)	Remove an element with shifting	$O(n)$
Deletion (End)	Remove the last element	$O(1)$
Searching (Linear)	Find an element sequentially	$O(n)$
Searching (Binary)	Find an element in a sorted array	$O(\log n)$
Updating	Modify an existing element	$O(1)$

Merging	Combine two arrays	$O(n + m)$
Reversing	Reverse the order of elements	$O(n)$

Applications of Array Operations

- Student Management Systems
 - Banking Applications
 - Hospital Management Systems
 - Inventory Management
 - Library Management Systems
 - Online Shopping Websites
 - Payroll Systems
 - Scientific Data Processing
-

Summary

- Array operations allow efficient manipulation of stored data.
- Traversal visits every element exactly once.
- Insertion and deletion require shifting elements and are generally $O(n)$.
- Updating an element by index is $O(1)$.
- Merging combines two arrays into one.
- Reversing changes the order of elements.
- Searching techniques vary in efficiency depending on the algorithm used.

2.4 Storage Representation of Arrays

Learning Objectives

After studying this section, students will be able to:

- Understand how arrays are stored in memory.
- Explain contiguous memory allocation.
- Calculate the address of an array element.
- Derive address calculation formulas.
- Understand row-major and column-major storage representations.
- Represent one-dimensional and multidimensional arrays in memory.

2.4.1 Introduction

An array stores elements in **contiguous (adjacent) memory locations**. Since the memory locations are continuous, the address of any element can be calculated directly using a mathematical formula. This direct address computation makes array access efficient.

Contiguous Memory Allocation

In an array, each element occupies consecutive memory locations.

Example:

Array A

Index

0 1 2 3 4

+-----+-----+-----+-----+-----+

| 15 | 25 | 35 | 45 | 55 |

+-----+-----+-----+-----+-----+

Memory Address

1000 1004 1008 1012 1016

Assuming:

- Base Address = 1000
- Size of each integer = 4 bytes

Each successive element occupies the next available memory location.

Advantages of Contiguous Memory

- Direct access to any element.
 - Constant-time access **O(1)**.
 - Efficient cache utilization.
 - Simple address calculation.
-

Limitations

- Fixed memory allocation in traditional arrays.
 - Memory expansion is difficult.
 - Insertion and deletion require shifting elements.
-

2.4.2 Address Calculation for One-Dimensional Arrays

The address of an element in a one-dimensional array is calculated using the following formula:

Where:

$$LOC(A[i]) = Base(A) + (i - LB) \times w$$

Symbol Meaning

LOC(A[i]) Address of element A[i]

Base(A) Base address of the array

i Index of the element

Symbol Meaning

LB	Lower bound of the array
w	Size of each element (bytes)

Zero-Based Indexing

For most programming languages such as Python, C, Java, and C++, the lower bound is **0**.

The formula becomes:

$$LOC(A[i]) = Base(A) + i \times w$$

Example 1

Given:

- Base Address = 1000
- Size of Integer = 4 bytes

Find the address of A[5].

Solution:

$$\begin{aligned} LOC(A[5]) &= 1000 + (5 \times 4) \\ &= 1000 + 20 \\ &= 1020 \end{aligned}$$

Therefore,

$$\boxed{LOC(A[5]) = 1020}$$

Example 2

Given:

Base Address = 500

Element Size = 2 Bytes

Find address of A[8].

Solution

$$\begin{aligned}LOC(A[8]) &= 500 + (8 \times 2) \\ &= 500 + 16 \\ &= 516\end{aligned}$$

Answer

Address = 516

Python Demonstration

base = 1000

size = 4

index = 5

address = base + index * size

print("Address =", address)

Output

Address = 1020

Complexity

Address calculation requires only one arithmetic operation.

Time Complexity

O(1)

2.4.3 Storage Representation of One-Dimensional Arrays

Consider

A = [10,20,30,40,50]

Memory Representation

Index

0 1 2 3 4

```
+-----+-----+-----+-----+-----+
| 10 | 20 | 30 | 40 | 50 |
+-----+-----+-----+-----+-----+
```

Address

1000 1004 1008 1012 1016

Each integer occupies 4 bytes.

Relationship Between Index and Address

Index Address

0	1000
1	1004
2	1008
3	1012
4	1016

Python Example

```
numbers = [10,20,30,40,50]
```

```
print(numbers[0])
```

```
print(numbers[4])
```

Output

10

50

2.4.4 Storage Representation of Two-Dimensional Arrays

A two-dimensional array consists of rows and columns.

Example

A[3][4]

Rows = 3

Columns = 4

Representation

Column

0 1 2 3

+---+---+---+---+

Row0 | 11 | 12 | 13 | 14 |

+---+---+---+---+

Row1 | 21 | 22 | 23 | 24 |

+---+---+---+---+

Row2 | 31 | 32 | 33 | 34 |

+---+---+---+---+

Memory Representation

Although a two-dimensional array appears as rows and columns, it is stored linearly in memory.

There are two storage methods:

- Row-Major Order
- Column-Major Order

Row-Major Order

In Row-Major Order, elements of each row are stored consecutively.

Example

Matrix

11 12 13

21 22 23

31 32 33

Memory Layout

11

↓

12

↓

13

↓

21

↓

22

↓

23

↓

31

↓

32

↓

33

Formula for Row-Major Order

$$LOC(A[i][j]) = Base + [(i \times N) + j] \times w$$

Where

Symbol	Meaning
N	Number of Columns
w	Size of Element

Example

Find address of

A[2][1]

Given

Base = 1000

Columns = 4

Size = 4 Bytes

Solution

$$\begin{aligned}LOC &= 1000 + [(2 \times 4) + 1] \times 4 \\&= 1000 + 36 \\&= 1036\end{aligned}$$

Answer

1036

Column-Major Order

In Column-Major Order, elements of each column are stored consecutively.

Example

Matrix

11 12 13

21 22 23

31 32 33

Memory

11

↓

21

↓

31

↓

12

↓

22

↓

32

↓

13

↓

23

↓

33

Formula for Column-Major Order

$$LOC = Base + [(j \times M) + i] \times w$$

Symbol Meaning

M Number of Rows

Difference Between Row-Major and Column-Major

Row-Major	Column-Major
Rows stored first	Columns stored first
Used by C, C++, Python (conceptually for nested lists)	Used by FORTRAN and MATLAB
Formula uses number of columns	Formula uses number of rows

Python Example

```
matrix = [  
  
    [10,20,30],  
  
    [40,50,60],  
  
    [70,80,90]  
  
]  
  
print(matrix[1][2])
```

Output

60

Key Points

- Arrays are stored in contiguous memory.
 - Direct address calculation enables constant-time access.
 - One-dimensional arrays use a simple address formula.
 - Two-dimensional arrays are physically stored in linear memory.
 - Row-major and column-major are two standard storage representations.
-

Summary

- Contiguous memory allocation is the basis of array implementation.
- Address calculation formulas allow direct access to elements.
- One-dimensional arrays require one index, while two-dimensional arrays require row and column indices.
- Row-major order stores row-wise, whereas column-major order stores column-wise.
- Address calculation is an **$O(1)$** operation.

2.5.1 Introduction

A **multidimensional array** is an array whose elements are themselves arrays. It is used to represent data in the form of **rows, columns, tables, matrices, or higher-dimensional structures**.

A one-dimensional array stores data in a single row, whereas a multidimensional array stores data in multiple dimensions.

Types of Multidimensional Arrays

1. Two-Dimensional Array (2D Array)
2. Three-Dimensional Array (3D Array)
3. n-Dimensional Array (nD Array)

2.5.2 Two-Dimensional Array (2D Array)

Definition

A **two-dimensional array** is an array of one-dimensional arrays. It is organized into **rows and columns** and is commonly used to represent matrices.

A 2D array requires **two indices**:

- Row Index
- Column Index

General notation:

A[row][column]

General Representation

Column

0 1 2

+-----+-----+-----+

Row 0 | 11 | 12 | 13 |

+-----+-----+-----+

Row 1 | 21 | 22 | 23 |

+-----+-----+-----+

Row 2 | 31 | 32 | 33 |

+-----+-----+-----+

Here,

- Number of Rows = 3
 - Number of Columns = 3
-

Declaration of a 2D Array

General Syntax

ArrayName[Rows][Columns]

Example

```
marks[5][4]
```

This array stores marks of **5 students** in **4 subjects**.

Initialization of a 2D Array in Python

```
matrix = [  
  
    [10,20,30],  
  
    [40,50,60],  
  
    [70,80,90]  
]
```

Representation

Row 0 → 10 20 30

Row 1 → 40 50 60

Row 2 → 70 80 90

Accessing Elements

General Syntax

```
array[row][column]
```

Example

```
matrix = [  
  
    [10,20,30],  
  
    [40,50,60],  
  
    [70,80,90]  
  
]  
  
print(matrix[1][2])
```

Output

60

Explanation

- Row = 1
- Column = 2

Element = 60

Traversing a 2D Array

A nested loop is required.

```
matrix = [  
  
    [10,20,30],  
  
    [40,50,60],  
  
    [70,80,90]  
  
]
```

```
for row in matrix:
```

```
    for value in row:
```



```
print(value, end=" ")
```

```
print()
```

Output

10 20 30

40 50 60

70 80 90

Complexity

Suppose

Rows = m

Columns = n

Total elements

$m \times n$

Traversal Complexity

$O(mn)$

Updating an Element

matrix = [

[10,20,30],

[40,50,60],

[70,80,90]

]

```
matrix[2][1] = 85
```

```
print(matrix)
```

Output

```
[[10,20,30],[40,50,60],[70,85,90]]
```

Complexity

O(1)

Addition of Two Matrices

```
A = [
```

```
    [1,2],
```

```
    [3,4]
```

```
]
```

```
B = [
```

```
    [5,6],
```

```
    [7,8]
```

```
]
```

```
result = [[0,0],[0,0]]
```

```
for i in range(2):
```

```
    for j in range(2):
```

```
        result[i][j] = A[i][j] + B[i][j]
```

```
print(result)
```

Output

[[6,8],[10,12]]

Complexity

$O(mn)$

Matrix Transpose

Transpose converts rows into columns.

Example

Original Matrix

1 2 3

4 5 6

Transpose

1 4

2 5

3 6

Python Program

```
matrix = [
```

```
    [1,2,3],
```

```
    [4,5,6]
```

```
]
```

```
transpose = []

for j in range(len(matrix[0])):

    row = []

    for i in range(len(matrix)):
        row.append(matrix[i][j])

    transpose.append(row)
```

```
print(transpose)
```

Output

```
[[1,4],[2,5],[3,6]]
```

Complexity

$O(mn)$

2.5.3 Three-Dimensional Array (3D Array)

Definition

A **three-dimensional array** consists of multiple two-dimensional arrays.

It requires **three indices**:

```
Array[layer][row][column]
```

Representation

Layer 0

1 2

3 4

Layer 1

5 6

7 8

Python Example

```
array3d = [
```

```
    [
```

```
        [1,2],
```

```
        [3,4]
```

```
    ],
```

```
    [
```

```
        [5,6],
```

```
        [7,8]
```

```
    ]
```

```
]
```

```
print(array3d[1][0][1])
```

Output

6

Traversing a 3D Array

for layer in array3d:

 for row in layer:

 for value in row:

 print(value)

Complexity

$O(lmn)$

where

- l = layers
- m = rows
- n = columns

2.5.4 n-Dimensional Arrays

Definition

An **n-dimensional array** is an array having **n indices**.

General representation

$A[i_1][i_2][i_3] \dots [i_n]$

Examples

- Image Processing
- Scientific Computing
- Machine Learning
- Numerical Simulations
- Computer Graphics

Address Calculation of n-Dimensional Arrays

In general, address calculation depends upon

- Number of dimensions
- Size of each dimension
- Element size
- Storage order

The detailed formula is beyond the scope of this unit unless specifically required.

Memory Representation

Regardless of the number of dimensions, arrays are stored linearly in memory.

For multidimensional arrays,

- Row-major order
- Column-major order

are used for storage.

Python Example

```
cube = [
```

```
    [
```

```
        [1,2],
```

```
        [3,4]
```

```
    ],
```

```
    [
```

```
        [5,6],
```

```
        [7,8]
```

```
]
```

```
]
```

```
print(cube)
```

Advantages of Multidimensional Arrays

- Suitable for matrix representation.
 - Easy representation of tabular data.
 - Efficient access using multiple indices.
 - Useful for mathematical computations.
 - Supports complex data organization.
-

Limitations

- Memory requirement increases with dimensions.
 - Traversal becomes more complex.
 - Insertion and deletion are expensive.
 - Address calculations become more complicated for higher dimensions.
-

Applications

- Matrix operations
- Image processing
- Computer graphics
- Scientific simulations
- Machine learning
- Database tables
- Game development

- Geographic Information Systems (GIS)

Difference Between 1D, 2D, and 3D Arrays

Feature	1D Array	2D Array	3D Array
Indices	1	2	3
Structure	Linear	Matrix	Collection of matrices
Example	[10,20,30]	[[1,2],[3,4]]	[[[1,2],[3,4]],[[5,6],[7,8]]]
Traversal	Single loop	Nested loops	Three nested loops
Complexity	$O(n)$	$O(mn)$	$O(lmn)$

2.6 Sparse Matrix Representation Using 2D

Learning Objectives

After studying this section, students will be able to:

- Define a sparse matrix.
 - Differentiate between dense and sparse matrices.
 - Explain the need for sparse matrix representation.
 - Represent sparse matrices using a two-dimensional (triplet) representation.
 - Convert a normal matrix into its sparse representation.
 - Write Python programs for sparse matrix representation.
 - Analyze the memory efficiency of sparse matrices.
-

2.6.1 Introduction

A **matrix** is a rectangular arrangement of elements organized into **rows and columns**.

Matrices are classified into two categories based on the number of zero elements:

- Dense Matrix
 - Sparse Matrix
-

2.6.2 Dense Matrix

A **Dense Matrix** is a matrix in which **most of the elements are non-zero**.

Example

5 2 7

1 8 4

6 9 3

Number of zeros = **0**

Hence, it is a **Dense Matrix**.

Characteristics of Dense Matrix

- Contains very few zero elements.
 - Most memory locations store useful data.
 - No special storage technique is required.
 - Stored directly as a 2D array.
-

2.6.3 Sparse Matrix

Definition

A **Sparse Matrix** is a matrix in which **most of the elements are zero**, and only a few elements are non-zero.

Generally,

If the number of zero elements is greater than the number of non-zero elements, the matrix is called a **Sparse Matrix**.

Example

0 0 0 8

0 0 0 0

5 0 0 0

0 0 9 0

Rows = 4

Columns = 4

Total Elements = 16

Non-zero Elements = 3

Zero Elements = 13

Since the number of zeros is much greater than the number of non-zero elements, it is a **Sparse Matrix**.

Characteristics of Sparse Matrix

- Majority of elements are zero.
 - Only a few elements contain useful data.
 - Saves memory by storing only non-zero elements.
 - Used in scientific and engineering applications.
-

2.6.4 Why Do We Need Sparse Matrix Representation?

When a sparse matrix is stored as a normal 2D array, memory is wasted because space is allocated for many zero values.

Example

0 0 0 5

0 0 0 0

8 0 0 0

0 0 2 0

Normal 2D storage requires memory for all **16 elements**, although only **3 elements** contain useful information.

Therefore, instead of storing every element, we store only the **non-zero elements** along with their positions.

This approach reduces memory usage.

Comparison: Normal Matrix Storage vs Sparse Matrix Storage

Normal Representation

+---+---+---+---+

| 0 | 0 | 0 | 5 |

+---+---+---+---+

| 0 | 0 | 0 | 0 |

+---+---+---+---+

| 8 | 0 | 0 | 0 |

+---+---+---+---+

| 0 | 0 | 2 | 0 |

+---+---+---+---+

Memory Required

16 Elements

Sparse Representation

Store only

- Row Number
- Column Number
- Value

Memory Required

Only 3 Records

2.6.5 Sparse Matrix Representation Using 2D (Triplet Representation)

The most common method is the **Triplet Representation**.

Each non-zero element is stored as

(Row, Column, Value)

The first row stores

(Number of Rows, Number of Columns, Number of Non-zero Elements)

Example Matrix

0 0 0 5

0 0 0 0

8 0 0 0

0 0 2 0

Rows = 4

Columns = 4

Non-zero Elements = 3

Step 1: Identify Non-zero Elements

Row	Column	Value
-----	--------	-------

0	3	5
---	---	---

2	0	8
---	---	---

3	2	2
---	---	---

Step 2: Create Triplet Representation

Row Column Value

4	4	3
0	3	5
2	0	8
3	2	2

Interpretation

First Row

4 4 3

Means

- Matrix has 4 rows
- Matrix has 4 columns
- Matrix contains 3 non-zero elements

Remaining rows contain Row Number, Column Number and Value

Memory Comparison

Suppose

Matrix Size

100×100

Total Elements

10000

Non-zero Elements

50

Normal Storage

10000 Elements

Sparse Storage

51 Records

(1 header row + 50 non-zero entries)

This significantly reduces memory usage.

Algorithm for Sparse Matrix Representation

Algorithm SparseMatrix(A, rows, cols)

Step 1: Count the number of non-zero elements.

Step 2: Store (rows, cols, count) as the first record.

Step 3: Traverse the matrix row by row.

Step 4: For every non-zero element,
store (row, column, value).

Step 5: Stop.

Python Program

```
matrix = [
```

```
    [0,0,0,5],
```

```
    [0,0,0,0],
```

```
    [8,0,0,0],
```

```
    [0,0,2,0]
```

```
]
```

```
rows = len(matrix)
```

```
cols = len(matrix[0])
```

```
triplet = []
```

```
count = 0

for i in range(rows):
    for j in range(cols):
        if matrix[i][j] != 0:
            count += 1

triplet.append([rows, cols, count])

for i in range(rows):
    for j in range(cols):
        if matrix[i][j] != 0:
            triplet.append([i, j, matrix[i][j]])

for row in triplet:
    print(row)
```

Output

[4, 4, 3]

[0, 3, 5]

[2, 0, 8]

[3, 2, 2]

Time Complexity

Let

- Number of Rows = m
- Number of Columns = n

The matrix must be traversed once.

Therefore,

Time Complexity = $O(m \times n)$

Space Complexity

Normal Matrix

$O(m \times n)$

Sparse Representation

$O(k)$

where

k = Number of Non-zero Elements

Advantages of Sparse Matrix Representation

- Reduces memory usage.
 - Stores only useful information.
 - Faster processing when the number of non-zero elements is small.
 - Efficient for large matrices with many zero values.
 - Easy to convert into other sparse formats.
-

Disadvantages

- Not suitable for dense matrices.
 - Additional processing is required for conversion.
 - Random access is slower compared to normal 2D arrays.
 - Some operations become more complex.
-

Applications of Sparse Matrices

- Graph representation using adjacency matrices.
- Image processing.
- Machine learning.
- Scientific computing.
- Geographic Information Systems (GIS).
- Finite element analysis.
- Recommendation systems.
- Network analysis.

Difference Between Dense Matrix and Sparse Matrix

Dense Matrix	Sparse Matrix
Most elements are non-zero	Most elements are zero
Requires more memory	Requires less memory
Stored directly as a 2D array	Stored using triplet representation
Faster random access	Better memory efficiency
Suitable for small or fully populated matrices	Suitable for large matrices with many zero elements

2.7 Searching Techniques

Learning Objectives

After studying this section, students will be able to:

- Understand the concept of searching.
- Differentiate between various searching techniques.
- Explain Sequential (Linear) Search.
- Write algorithms and Python programs for Linear Search.

- Analyze the time complexity of Linear Search.
 - Understand the advantages and limitations of Linear Search.
-

2.7.1 Introduction to Searching

Searching is one of the most fundamental operations in Data Structures.

Searching is the process of locating a particular element (called the **key**) in a collection of data and determining whether it exists or not.

If the element is found, its position (index) is returned; otherwise, the search reports that the element is not present.

Need for Searching

Searching is required in almost every software application to retrieve information quickly.

Searching is performed to:

- Find a specific record.
 - Verify whether an element exists.
 - Retrieve the position (index) of an element.
 - Update or delete a specific record after locating it.
-

Types of Searching Techniques

Linear Search Binary Search Fibonacci Search Indexed Sequential Search

In this section, we study **Linear (Sequential) Search** in detail.

2.7.2 Sequential (Linear) Search

Definition

Sequential Search, also called **Linear Search**, is the simplest searching technique in which the elements are examined **one by one from the beginning until the required element is found or the end of the array is reached**.

No sorting of the array is required.

Working Principle

The algorithm compares the search key with every element of the array.

- If a match is found, the index is returned.
 - If all elements are checked and no match is found, the search terminates.
-

Example

Consider the following array:

Index

0 1 2 3 4 5

```
+---+---+---+---+---+
| 15 | 25 | 35 | 45 | 55 | 65 |
+---+---+---+---+---+
```

Search Key = **45**

Comparisons performed:

15 X

↓

25 X

↓

35 X

↓

45 ✓

Element found at **Index 3**.

Algorithm

Algorithm LinearSearch(A, n, key)

Input:

A → Array

n → Number of elements

key → Element to be searched

Output:

Position of the element or -1

Step 1:

For i = 0 to n-1

Step 2:

If A[i] == key

Step 3:

Return i

Step 4:

End If

Step 5:

End For

Step 6:

Return -1

Step 7:

Stop

Python Program

```
def linear_search(arr, key):  
  
    for i in range(len(arr)):  
  
        if arr[i] == key:  
            return i  
  
    return -1  
  
numbers = [15, 25, 35, 45, 55, 65]  
  
key = 45  
  
position = linear_search(numbers, key)  
  
if position != -1:  
    print("Element Found at Index", position)  
else:  
    print("Element Not Found")
```

Output

Element Found at Index 3

Example 2

Searching an element that is not present.

```
numbers = [12, 24, 36, 48, 60]
```

```
key = 50
```

```
Comparisons
```

```
12 X
```

```
24 X
```

36 X

48 X

60 X

Output

Element Not Found

Complexity Analysis

Suppose the array contains **n elements**.

Best Case

The element is found at the first position.

Comparisons = 1

$T(n)=1$

Time Complexity

$O(1)$

Average Case

The element is found somewhere in the middle.

Average comparisons

$2n + 1$

Time Complexity

$O(n)$

Worst Case

The element is at the last position or not present.

Comparisons

n

Time Complexity

$O(n)$

Space Complexity

Linear Search uses only one additional variable.

Therefore,

Space Complexity = $O(1)$

Complexity Summary

Case	Comparisons	Time Complexity
Best	1	$O(1)$
Average	$(n + 1) / 2$	$O(n)$
Worst	n	$O(n)$

Advantages of Linear Search

- Very easy to understand and implement.
 - Works on both sorted and unsorted arrays.
 - No preprocessing or sorting is required.
 - Suitable for small datasets.
 - Requires constant extra memory.
-

Limitations of Linear Search

- Inefficient for large datasets.
- Searches elements one by one.
- Performance decreases as the number of elements increases.

- Slower than Binary Search for sorted arrays.
-

Applications of Linear Search

- Searching in small arrays.
 - Searching in unsorted data.
 - Finding records in simple applications.
 - Checking whether an element exists in a list.
 - Searching in linked lists where binary search is not applicable.
-

Linear Search vs Binary Search

Feature	Linear Search	Binary Search
Data Requirement	Sorted or Unsorted	Sorted Only
Searching Method	Sequential	Divide and Conquer
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$
Suitable For	Small datasets	Large sorted datasets

2.7.3 Binary Search

Learning Objectives

After studying this section, students will be able to:

- Define Binary Search.
- Understand the prerequisite for Binary Search.
- Explain the working principle of Binary Search.
- Write the Binary Search algorithm.

- Implement Binary Search in Python.
 - Perform a dry run of the algorithm.
 - Analyze the time and space complexity.
 - Compare Binary Search with Linear Search.
-

2.7.3.1 Introduction

Binary Search is one of the most efficient searching algorithms used to search an element in a **sorted array**.

Instead of checking every element one by one, Binary Search repeatedly divides the search space into two equal halves until the desired element is found.

Because it reduces the search space by half after every comparison, Binary Search is much faster than Linear Search for large datasets.

Prerequisite

Binary Search works only on a sorted array.

If the array is unsorted, it must first be sorted before Binary Search can be applied.

Example of Sorted Array

Index

0 1 2 3 4 5 6

+---+---+---+---+---+---+---+

|10 |20 |30 |40 |50 |60 |70 |

+---+---+---+---+---+---+---+

Working Principle

Binary Search follows these steps:

1. Find the middle element of the array.
 2. Compare the middle element with the search key.
 3. If they are equal, the search is successful.
 4. If the search key is smaller, continue searching in the left half.
 5. If the search key is larger, continue searching in the right half.
 6. Repeat the process until the element is found or the search space becomes empty.
-

Binary Search Algorithm

Algorithm BinarySearch(A, low, high, key)

Input:

A → Sorted Array

low → Starting index

high → Last index

key → Element to search

Step 1:

While $low \leq high$

Step 2:

$mid = (low + high) // 2$

Step 3:

If $A[mid] == key$

Return mid

Step 4:

Else If $key < A[mid]$

high = mid - 1

Step 5:

Else

low = mid + 1

Step 6:

End While

Step 7:

Return -1

Step 8:

Stop

Example

Search **50** in the following sorted array.

Index

0 1 2 3 4 5 6

10 20 30 40 50 60 70

Iteration 1

low = 0

high = 6

mid = $(0+6)//2 = 3$

A[3] = 40

Since

$50 > 40$

Search the right half.

Iteration 2

low = 4

high = 6

mid = $(4+6)//2 = 5$

A[5] = 60

Since

$50 < 60$

Search the left half.

Iteration 3

low = 4

high = 4

mid = 4

A[4] = 50

Element Found.

Index = 4

Python Program (Iterative Binary Search)

```
def binary_search(arr, key):
```

```
    low = 0
```

```
    high = len(arr) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] == key:
```

```
            return mid
```

```
        elif key < arr[mid]:
```

```
            high = mid - 1
```

```
        else:
```

```
            low = mid + 1
```

```
    return -1
```

```
numbers = [10,20,30,40,50,60,70]
```

```
key = 50
```

```
position = binary_search(numbers, key)

if position != -1:
    print("Element Found at Index", position)
else:
    print("Element Not Found")
```

Output

Element Found at Index 4

Python Program (Recursive Binary Search)

```
def binary_search(arr, low, high, key):

    if low > high:
        return -1

    mid = (low + high) // 2

    if arr[mid] == key:
        return mid

    elif key < arr[mid]:
        return binary_search(arr, low, mid - 1, key)

    else:
        return binary_search(arr, mid + 1, high, key)
```

```
numbers = [10,20,30,40,50,60,70]
```

```
result = binary_search(numbers, 0, len(numbers)-1, 50)
```

```
if result != -1:
```

```
    print("Element Found at Index", result)
```

```
else:
```

```
    print("Element Not Found")
```

Output

Element Found at Index 4

Complexity Analysis

Suppose the array contains **n** elements.

Each iteration reduces the search space by half.

Number of comparisons:

n

n/2

n/4

n/8

...

1

After **k** divisions,

$$\frac{n}{2^k} = 1$$

Therefore,

$$2^k = n$$

Taking logarithm on both sides,

$$= \log n$$

Hence,

Best Case

The middle element is the search key.

Time Complexity = $O(1)$

Average Case

Time Complexity = $O(\log n)$

Worst Case

Time Complexity = $O(\log n)$

Space Complexity

Iterative Binary Search

Only a few variables are used.

Space Complexity = $O(1)$

Recursive Binary Search

The recursion stack requires additional memory.

Space Complexity = $O(\log n)$

Complexity Summary

Case	Time Complexity
------	-----------------

Best	$O(1)$
Average	$O(\log n)$
Worst	$O(\log n)$
Iterative Space	$O(1)$
Recursive Space	$O(\log n)$

Advantages

- Very efficient for large datasets.
 - Requires fewer comparisons than Linear Search.
 - Logarithmic time complexity.
 - Suitable for sorted arrays.
 - Widely used in searching applications.
-

Limitations

- Works only on sorted data.
 - Not suitable for linked lists due to lack of direct indexing.
 - Sorting overhead is required if the data is unsorted.
 - Recursive implementation uses additional stack memory.
-

Linear Search vs Binary Search

Feature	Linear Search	Binary Search
Data Requirement	Sorted/Unsorted	Sorted Only
Method	Sequential Search	Divide Search Space
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$

Worst Case	$O(n)$	$O(\log n)$
Suitable for Small Data	Yes	Yes
Suitable for Large Data	No	Yes

Applications of Binary Search

- Searching in sorted arrays.
 - Dictionary and phone directory lookup.
 - Database indexing.
 - Library management systems.
 - Searching in large datasets.
 - Searching in sorted files.
-

Summary

- Binary Search is an efficient searching algorithm for sorted arrays.
- It repeatedly divides the search space into two halves.
- It requires the input array to be sorted.
- Iterative implementation uses constant extra memory.
- Binary Search is significantly faster than Linear Search for large datasets.

2.7.4 Fibonacci Search

Learning Objectives

After studying this section, students will be able to:

- Define Fibonacci Search.
- Understand the Fibonacci number series.
- Explain the working principle of Fibonacci Search.

- Write the algorithm for Fibonacci Search.
- Implement Fibonacci Search in Python.
- Perform a dry run of the algorithm.
- Analyze the time and space complexity.
- Compare Fibonacci Search with Binary Search.

2.7.4.1 Introduction

Fibonacci Search is an efficient searching algorithm used to search an element in a **sorted array**. It is based on the **Fibonacci number sequence** and uses Fibonacci numbers to divide the search range.

Unlike Binary Search, which divides the array into two equal halves, Fibonacci Search divides the array into sections based on Fibonacci numbers.

This algorithm is particularly useful for searching data stored in sequential storage devices because it reduces the number of expensive division operations.

Prerequisite

The array must be sorted in ascending order.

Example

Index

```

0   1   2   3   4   5   6   7

+---+---+---+---+---+---+---+
|10 |20 |30 |40 |50 |60 |70 |80 |
+---+---+---+---+---+---+---+

```

Fibonacci Number Series

The Fibonacci sequence is generated using the following rule:

$$F(n)=F(n-1)+F(n-2) \quad F(n)=F(n-1)+F(n-2) \quad F(n)=F(n-1)+F(n-2)$$

where

- $F(0)=0$
- $F(1)=1$

Sequence:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Each number is the sum of the previous two numbers.

Working Principle

Fibonacci Search works as follows:

1. Find the smallest Fibonacci number greater than or equal to the size of the array.
 2. Compare the key with the element at the calculated index.
 3. If the key is smaller, search the left portion.
 4. If the key is larger, search the right portion.
 5. Continue reducing the search range using Fibonacci numbers until the element is found or the search space becomes empty.
-

Algorithm

Algorithm FibonacciSearch(A, n, key)

Input:

A → Sorted Array

n → Number of elements

key → Element to search

Step 1:

Generate the smallest Fibonacci number greater than or equal to n.

Step 2:

Initialize offset = -1.

Step 3:

While Fibonacci number > 1

$i = \min(\text{offset} + \text{fibMMm2}, n-1)$

If $A[i] < \text{key}$

Move search to right.

Update Fibonacci numbers.

Else if $A[i] > \text{key}$

Move search to left.

Update Fibonacci numbers.

Else

Return i

Step 4:

If last remaining element equals key

Return its index.

Step 5:

Return -1.

Step 6:

Stop.

Example

Search **60** in the sorted array.

Index

0 1 2 3 4 5 6 7

10 20 30 40 50 60 70 80

Suppose the smallest Fibonacci number greater than or equal to 8 is:

8

The algorithm selects indices according to Fibonacci values until the required element is located.

Result

Element Found at Index 5

Dry Run Table

Iteration	Fibonacci Number	Index Checked	Element	Result
1	8	2	30	Search Right
2	5	4	50	Search Right
3	3	5	60	Found

Python Program

```
def fibonacci_search(arr, key):
```

```
    n = len(arr)
```

```
    fib2 = 0
```

```
    fib1 = 1
```

```
    fib = fib1 + fib2
```

```
    while fib < n:
```

```
        fib2 = fib1
```

```
        fib1 = fib
```

```
        fib = fib1 + fib2
```

```
offset = -1
```

```
while fib > 1:
```

```
    i = min(offset + fib2, n - 1)
```

```
    if arr[i] < key:
```

```
        fib = fib1
```

```
        fib1 = fib2
```

```
        fib2 = fib - fib1
```

```
        offset = i
```

```
    elif arr[i] > key:
```

```
        fib = fib2
```

```
        fib1 = fib1 - fib2
```

```
        fib2 = fib - fib1
```

```
    else:
```

```
        return i
```

```
if fib1 and offset + 1 < n and arr[offset + 1] == key:
```

```
    return offset + 1
```

```
return -1
```

```
numbers = [10,20,30,40,50,60,70,80]
```

```
key = 60
```

```
result = fibonacci_search(numbers, key)
```

```
if result != -1:
```

```
    print("Element Found at Index", result)
```

```
else:
```

```
    print("Element Not Found")
```

Output

Element Found at Index 5

Complexity Analysis

Suppose the array contains n elements.

The search space decreases according to Fibonacci numbers, resulting in logarithmic performance.

Best Case

The element is found in the first comparison.

Time Complexity = $O(1)$

Average Case

Time Complexity = $O(\log n)$

Worst Case

Time Complexity = $O(\log n)$

Space Complexity

The iterative algorithm uses only a fixed number of variables.

Space Complexity = $O(1)$

Complexity Summary

Case	Time Complexity
Best	$O(1)$
Average	$O(\log n)$
Worst	$O(\log n)$

Space	$O(1)$
-------	--------

Advantages

- Efficient for large sorted arrays.
- Does not require division operations.
- Uses only addition and subtraction.
- Suitable for sequential memory access.
- Constant extra memory requirement.

Limitations

- Works only on sorted arrays.
- More complex than Linear Search.
- Slightly more difficult to implement than Binary Search.
- Less commonly used in modern applications.

Binary Search vs Fibonacci Search

Feature	Binary Search	Fibonacci Search
Data Requirement	Sorted Array	Sorted Array
Division Required	Yes	No
Based On	Midpoint	Fibonacci Numbers
Best Case	$O(1)$	$O(1)$
Average Case	$O(\log n)$	$O(\log n)$
Worst Case	$O(\log n)$	$O(\log n)$
Space Complexity	$O(1)$ (Iterative)	$O(1)$

Applications

- Searching in sorted arrays.
- Large sequential storage systems.
- Database indexing.
- Embedded systems.
- Applications where division operations are expensive.

2.7.5 Indexed Sequential Search

Learning Objectives

After studying this section, students will be able to:

- Define Indexed Sequential Search.
 - Understand the concept of indexing.
 - Explain the working principle of Indexed Sequential Search.
 - Create an index table.
 - Write the algorithm for Indexed Sequential Search.
 - Implement Indexed Sequential Search in Python.
 - Analyze the time and space complexity.
 - Compare Indexed Sequential Search with other searching techniques.
-

2.7.5.1 Introduction

Indexed Sequential Search (ISS) is a searching technique that combines the advantages of **Sequential Search** and **Indexing**.

Instead of searching every element one by one, an **index table** is first searched to identify the block in which the required element may exist. A **Linear Search** is then performed only within that block.

This approach significantly reduces the number of comparisons compared to a simple Linear Search.

Prerequisites

Before applying Indexed Sequential Search:

- Data must be **sorted**.
 - Data should be divided into **blocks**.
 - An **index table** must be created.
-

Basic Concept

Indexed Sequential Search works in **two phases**:

Phase 1

Search the **Index Table**.

↓

Phase 2

Perform **Linear Search** only in the identified block.

This makes searching faster than scanning the entire array.

Example

Suppose we have the following sorted array.

Index

0 1 2 3 4 5 6 7 8 9

10 15 20 25 30 35 40 45 50 55

Divide the array into blocks of size **2**.

Block Representation

Block 1

10 15

Block 2

20 25

Block 3

30 35

Block 4

40 45

Block 5

50 55

Index Table

The first element of each block is stored in an index table.

Block	Starting Value	Starting Index
1	10	0
2	20	2

3	30	4
4	40	6
5	50	8

Searching Example

Search Key

45

Step 1

Search the index table.

10

20

30

40

50

Since

$40 \leq 45 < 50$

The required element lies in **Block 4**.

Step 2

Search only Block 4.

40

45

Element Found.

Index = **7**

Working Principle

1. Divide the sorted array into blocks.
2. Store the first element of every block in the index table.
3. Search the index table.

4. Identify the correct block.
 5. Perform Linear Search within that block.
 6. Return the position if found.
 7. Otherwise return -1.
-

Algorithm

Algorithm IndexedSequentialSearch(A, IndexTable, key)

Input

A → Sorted Array

IndexTable → Index values

key → Search Element

Step 1

Search Index Table.

Step 2

Identify the correct block.

Step 3

Perform Linear Search within that block.

Step 4

If key is found

Return Index.

Else

Return -1.

Step 5

Stop.

Python Program

```
def indexed_sequential_search(arr, key, block_size):
```

```
    n = len(arr)
```

```
    # Create index table
```

```
    index = []
```

```
    for i in range(0, n, block_size):
```

```
        index.append((arr[i], i))
```

```
    # Find appropriate block
```

```
    start = 0
```

```
    end = n
```

```
    for i in range(len(index)):
```

```
        if key < index[i][0]:
```

```
            break
```

```
        start = index[i][1]
```

```
    end = min(start + block_size, n)
```

```
    # Linear Search inside the block
```

```
    for i in range(start, end):
```

```
        if arr[i] == key:
```

```
            return i
```

```
    return -1
```

```
numbers = [10,15,20,25,30,35,40,45,50,55]
```

```
key = 45
```

```
result = indexed_sequential_search(numbers, key, 2)
```

```
if result != -1:
```

```
    print("Element Found at Index", result)
```

```
else:
```

```
    print("Element Not Found")
```

Output

Element Found at Index 7

Complexity Analysis

Suppose

n = Number of Elements

b = Block Size

Searching involves two operations:

- Searching the index table.
- Searching within one block.

Best Case

Element found at the beginning of the selected block.

Time Complexity = $O(1)$

Average Case

Time Complexity = $O(\sqrt{n})$

(when the block size is chosen optimally, approximately $\sqrt{n}$)

Worst Case

Searching the index table and then scanning one complete block.

Time Complexity = $O(\sqrt{n})$

Space Complexity

Additional memory is required for storing the index table.

Space Complexity = $O(\sqrt{n})$

Complexity Summary

Case	Time Complexity
Best	$O(1)$
Average	$O(\sqrt{n})$
Worst	$O(\sqrt{n})$
Space	$O(\sqrt{n})$

Advantages

- Faster than Linear Search for large sorted datasets.
 - Reduces the number of comparisons.
 - Easy to implement.
 - Suitable for sequential file organization.
 - Efficient for moderately large datasets.
-

Limitations

- Works only on sorted data.

- Additional memory is required for the index table.
 - Updating the data requires updating the index table.
 - Less efficient than Binary Search for in-memory arrays.
-

Applications

- Indexed sequential file systems.
- Library management systems.
- Banking databases.
- Student record management.

2.8 Introduction to Sorting

Learning Objectives

After studying this section, students will be able to:

- Define sorting.
 - Explain the need for sorting.
 - Understand the characteristics of a good sorting algorithm.
 - Differentiate between ascending and descending order.
 - Explain the concepts of stability, efficiency, and number of passes.
 - Differentiate between internal and external sorting.
 - Identify applications of sorting.
-

2.8.1 Introduction

Sorting is one of the most important operations in Data Structures.

Sorting is the process of arranging data elements in a specific order, usually **ascending** or **descending**.

A sorted collection of data makes searching, updating, merging, and analyzing information much easier and faster.

Sorting is widely used in computer applications such as databases, operating systems, file management, and data analysis.

Definition

Sorting is the process of arranging elements of a collection into a specified order (ascending or descending) based on one or more keys.

Need for Sorting

Sorting is performed because it:

- Makes searching faster.
 - Organizes data systematically.
 - Simplifies data analysis.
 - Helps in efficient report generation.
 - Improves the performance of many algorithms.
 - Makes duplicate detection easier.
 - Facilitates data merging.
-

Example

Unsorted Data

45 12 89 34 25 67

Ascending Order

12 25 34 45 67 89

Descending Order

89 67 45 34 25 12

Types of Sorting

Sorting can be classified into two categories.

1. Ascending Order

The elements are arranged from the **smallest value to the largest value**.

Example

5 10 18 25 30 45

2. Descending Order

The elements are arranged from the **largest value to the smallest value**.

Example

45 30 25 18 10 5

Characteristics of a Good Sorting Algorithm

A good sorting algorithm should possess the following characteristics:

- Correctly sort all elements.
 - Produce consistent results.
 - Require minimum execution time.
 - Use minimum memory.
 - Be easy to understand and implement.
 - Perform efficiently for both small and large datasets.
 - Handle duplicate values correctly.
-

Classification of Sorting Algorithms

Sorting algorithms are classified based on different criteria.

Sorting Algorithms

Internal Sorting

External Sorting

Sorting algorithms can also be classified as:

- Comparison-based sorting
 - Non-comparison sorting
 - Stable sorting
 - Unstable sorting
 - In-place sorting
 - Out-of-place sorting
-

Comparison-Based Sorting

These algorithms compare two elements to determine their order.

Examples:

- Bubble Sort
 - Selection Sort
 - Insertion Sort
 - Merge Sort
 - Quick Sort
 - Heap Sort
-

Non-Comparison Sorting

These algorithms do not compare elements directly.

Examples:

- Counting Sort

- Radix Sort
- Bucket Sort

(These algorithms are beyond the scope of the SPPU Unit II syllabus.)

Stable and Unstable Sorting

Sorting algorithms can also be classified according to whether they preserve the relative order of duplicate elements.

Stable Sorting



Relative order of equal elements is preserved.

Unstable Sorting



Relative order of equal elements may change.

The concepts of **Stability** will be discussed in detail in the next section.

In-place and Out-of-place Sorting

In-place Sorting

Sorting is performed within the original array using very little extra memory.

Examples

- Bubble Sort
 - Selection Sort
 - Insertion Sort
 - Quick Sort
-

Out-of-place Sorting

Sorting requires additional memory.

Examples

- Merge Sort

General Steps of Sorting

Every sorting algorithm follows the same basic procedure.

Input Unsorted Data

↓

Compare Elements

↓

Rearrange Elements

↓

Repeat Comparisons

↓

Sorted Data

Python Example

Using Python's Built-in `sort()` Method

```
numbers = [45, 12, 89, 34, 25, 67]
```

```
numbers.sort()
```

```
print(numbers)
```

Output

[12, 25, 34, 45, 67, 89]

Descending Order

numbers = [45, 12, 89, 34, 25, 67]

numbers.sort(reverse=True)

print(numbers)

Output

[89, 67, 45, 34, 25, 12]

Complexity of Sorting

The efficiency of a sorting algorithm is measured using:

- Time Complexity
- Space Complexity

Some common complexities are:

Complexity	Description
$O(n^2)$	Suitable for small datasets
$O(n \log n)$	Suitable for large datasets
$O(n)$	Best possible for special cases

The detailed complexity of each sorting algorithm will be discussed in later sections.

Applications of Sorting

Sorting is used in many computer applications such as:

- Database management systems.
- File organization.

- Search optimization.
 - Report generation.
-

Advantages of Sorting

- Faster searching.
 - Better organization of data.
 - Easier maintenance of records.
 - Simplifies duplicate detection.
 - Improves algorithm efficiency.
-

Limitations

- Sorting large datasets may require significant processing time.
- Some algorithms require extra memory.
- Inefficient algorithms become slow for very large datasets.

2.8.2 Stability of Sorting Algorithms

Learning Objectives

After studying this section, students will be able to:

- Define stability in sorting algorithms.
 - Understand the importance of stable sorting.
 - Differentiate between stable and unstable sorting algorithms.
 - Identify stable and unstable sorting techniques.
 - Analyze the advantages and disadvantages of stable sorting.
 - Compare different sorting algorithms based on stability.
-

2.8.2.1 Introduction

When sorting data, it is common to encounter **duplicate values (equal keys)**. In such cases, it is important to know whether the sorting algorithm **preserves the original order of equal elements**.

This property is called **Stability**.

Stability becomes important when sorting records that contain **multiple fields**, such as student records, employee records, product details, and examination results.

Definition

A sorting algorithm is said to be Stable if it preserves the relative order of elements having equal keys after sorting.

If the order changes after sorting, the algorithm is called **Unstable**.

Understanding Stability

Consider the following student records sorted according to **Marks**.

Before Sorting

Student	Marks
Amit	80
Priya	70
Rahul	80
Sneha	60

Notice that:

- Amit scored **80**
- Rahul also scored **80**
- Amit appears before Rahul.

After Stable Sorting (Ascending)

Student	Marks
Sneha	60
Priya	70
Amit	80
Rahul	80

The order of students with **80 marks** remains unchanged.

Therefore,

Stable Sorting ✓

After Unstable Sorting (Ascending)

Student	Marks
Sneha	60
Priya	70
Rahul	80
Amit	80

Here,

Rahul appears before Amit.

Their original order has changed.

Therefore,

Unstable Sorting ✗

Visual Representation

Original Order

Amit (80)

↓

Rahul (80)

Stable Sorting

Amit (80)

↓

Rahul (80)

Order remains the same.

Unstable Sorting

Rahul (80)



Amit (80)

Order changes.

Why is Stability Important?

Stability is useful when data contains **multiple attributes**.

Suppose employee records are first sorted according to **Department**, and then sorted according to **Salary**.

If the second sorting algorithm is stable, employees with the same salary will still remain grouped according to their department.

Thus, stable sorting preserves the ordering established by previous sorting operations.

Example with Multiple Fields

Original Records

Roll No	Name	Marks
101	Amit	85
102	Neha	90
103	Rahul	85
104	Priya	70

Sorting by **Marks** using a **Stable Sorting Algorithm** gives:

Roll No	Name	Marks
104	Priya	70
101	Amit	85

103	Rahul	85
102	Neha	90

Notice that Amit and Rahul both have **85 marks**, and their original order is maintained.

Stable Sorting Algorithms

The following sorting algorithms preserve the order of equal elements.

Sorting Algorithm	Stable
Bubble Sort	✓ Yes
Insertion Sort	✓ Yes
Merge Sort	✓ Yes
Counting Sort	✓ Yes
Radix Sort	✓ Yes

Unstable Sorting Algorithms

The following algorithms may change the order of equal elements.

Sorting Algorithm	Stable
Selection Sort	✗ No
Quick Sort	✗ No
Heap Sort	✗ No
Shell Sort	✗ No

Python Demonstration of Stable Sorting

Python's built-in `sorted()` function uses **Timsort**, which is a **stable sorting algorithm**.

```
students = [
```

```
    ("Amit", 80),
```

```
("Priya", 70),  
  
("Rahul", 80),  
  
("Sneha", 60)  
]  
  
sorted_students = sorted(students, key=lambda x: x[1])
```

```
for student in sorted_students:  
    print(student)
```

Output

```
('Sneha', 60)
```

```
('Priya', 70)
```

```
('Amit', 80)
```

```
('Rahul', 80)
```

Notice that **Amit** remains before **Rahul**, preserving the original order for equal marks.

Advantages of Stable Sorting

- Preserves the relative order of equal elements.
- Suitable for sorting records with multiple fields.
- Produces predictable and consistent results.
- Useful in database applications and record management.
- Maintains previous sorting order when multiple sorts are applied.

Limitations of Stable Sorting

- Some stable algorithms require additional memory.
- Stable algorithms may not always be the fastest.
- Achieving stability can increase implementation complexity.

Difference Between Stable and Unstable Sorting

Stable Sorting	Unstable Sorting
Preserves the order of equal elements	Does not preserve the order of equal elements
Suitable for multi-key sorting	Not suitable for preserving previous order
Predictable results	Order of equal elements may change
Examples: Bubble, Insertion, Merge	Examples: Selection, Quick, Heap

2.8.3 Efficiency of Sorting Algorithms

Learning Objectives

After studying this section, students will be able to:

- Define the efficiency of a sorting algorithm.
 - Understand the factors affecting sorting performance.
 - Explain Time Complexity and Space Complexity.
 - Analyze the efficiency of different sorting algorithms.
 - Compare sorting algorithms based on their performance.
 - Select an appropriate sorting algorithm for a given problem.
-

2.8.3.1 Introduction

When multiple sorting algorithms are available, an important question arises:

Which sorting algorithm should be used?

The answer depends on the **efficiency** of the sorting algorithm.

The efficiency of a sorting algorithm is measured by the amount of:

- Time required to sort the data.
- Memory required during sorting.

An efficient sorting algorithm should complete the sorting process quickly while using minimum memory.

Definition

Efficiency of a sorting algorithm refers to its performance in terms of execution time and memory usage while sorting a collection of data.

Efficiency is mainly measured using:

- **Time Complexity**
 - **Space Complexity**
-

Factors Affecting Efficiency

The performance of a sorting algorithm depends on several factors:

- Number of elements (n)
 - Initial arrangement of data
 - Sorting technique used
 - Available memory
 - Hardware performance
 - Programming language implementation
-

Time Complexity

Definition

Time Complexity is the amount of time required by an algorithm to complete its execution as the size of the input increases.

It is expressed using **Asymptotic Notations**, such as:

- Big O (O)
- Omega (Ω)
- Theta (Θ)

Cases of Time Complexity

1. Best Case

The minimum amount of time required.

Example:

A Bubble Sort where the array is already sorted.

2. Average Case

The average execution time for random input.

3. Worst Case

The maximum amount of time required.

Example:

A reverse sorted array for Bubble Sort.

Example

Suppose we have

5 Numbers

Sorting may take

20 Comparisons

If

1000 Numbers

Sorting may require

Thousands of Comparisons

Thus, execution time increases with input size.

Common Time Complexities

Complexity	Performance
$O(1)$	Excellent
$O(\log n)$	Very Fast
$O(n)$	Fast
$O(n \log n)$	Efficient
$O(n^2)$	Slow for large data
$O(n^3)$	Very Slow
$O(2^n)$	Extremely Slow

As the input size increases,

- $O(n \log n)$ grows slowly.
- $O(n^2)$ grows rapidly.

Therefore,

Algorithms having **$O(n \log n)$** are generally preferred over **$O(n^2)$** .

Space Complexity

Definition

Space Complexity is the amount of additional memory required by an algorithm during execution.

It includes

- Temporary variables
 - Auxiliary arrays
 - Recursive function calls
-

Example

Bubble Sort

Requires only

One temporary variable

Space Complexity

$O(1)$

Merge Sort

Requires

An additional array

Space Complexity

$O(n)$

Comparison of Time and Space Complexity

Sorting Algorithm	Best	Average	Worst	Space
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

Efficiency Comparison

Bubble Sort

Advantages

- Very simple
- Suitable for small datasets

Disadvantages

- Slow for large datasets

Selection Sort

Advantages

- Performs fewer swaps
- Simple implementation

Disadvantages

- Comparisons remain high
-

Insertion Sort

Advantages

- Efficient for nearly sorted arrays
- Stable sorting

Disadvantages

- Inefficient for large datasets
-

Merge Sort

Advantages

- Predictable performance
- Fast for large datasets

Disadvantages

- Requires extra memory
-

Quick Sort

Advantages

- Very fast in practice
- Widely used

Disadvantages

- Worst case $O(n^2)$
- Recursive implementation

Choosing an Efficient Sorting Algorithm

Situation	Recommended Algorithm
Small dataset	Bubble Sort, Insertion Sort
Nearly sorted data	Insertion Sort
Large dataset	Merge Sort
Fast practical sorting	Quick Sort
Memory is limited	Selection Sort, Bubble Sort

Performance Comparison Chart

Algorithm	Comparisons	Memory	Stable
Bubble Sort	High	Low	Yes
Selection Sort	High	Low	No
Insertion Sort	Medium	Low	Yes
Merge Sort	Low	High	Yes
Quick Sort	Very Low	Medium	No

Importance of Efficiency

Efficient sorting algorithms

- Reduce execution time.
- Improve application performance.
- Save computational resources.
- Process large datasets quickly.

- Improve responsiveness in software systems.
-

Applications

Efficiency analysis helps in

- Database management
- Scientific computing
- Artificial Intelligence
- Operating Systems
- Data Analytics
- Machine Learning
- Search Engines
- Cloud Computing

2.8.4 Number of Passes in Sorting Algorithms

Learning Objectives

After studying this section, students will be able to:

- Define the term **Pass** in a sorting algorithm.
 - Understand why multiple passes are required during sorting.
 - Explain the number of passes in different sorting algorithms.
 - Calculate the number of passes required for a given array.
 - Compare sorting algorithms based on the number of passes.
-

2.8.4.1 Introduction

Sorting algorithms do not usually arrange all elements correctly in a single step. Instead, they repeatedly scan the array and gradually move elements to their correct positions.

Each complete traversal of the array is called a **Pass**.

The total number of passes required depends on:

- The sorting algorithm used.
 - The number of elements.
 - The initial arrangement of the data.
-

Definition

A Pass is one complete traversal (or iteration) over the array during the sorting process, where comparisons and/or swaps are performed.

During each pass, one or more elements are moved closer to their correct positions.

Why are Multiple Passes Required?

In most sorting algorithms, a single pass is not sufficient to sort all elements because:

- Only a few elements reach their correct positions in each pass.
 - Remaining elements are sorted in subsequent passes.
 - Repeated comparisons gradually arrange the data in the correct order.
-

Example

Consider the following unsorted array.

50 20 40 10 30

Pass 1

Compare adjacent elements and swap if required.

50 20 40 10 30

↓

20 50 40 10 30

↓

20 40 50 10 30

↓

20 40 10 50 30

↓

20 40 10 30 50

After Pass 1

20 40 10 30 50

Notice that **50** has reached its correct (last) position.

Pass 2

20 40 10 30 50

↓

20 40 10 30 50

↓

20 10 40 30 50

↓

20 10 30 40 50

After Pass 2

20 10 30 40 50

Now **40** is in its correct position.

Pass 3

20 10 30 40 50

↓

10 20 30 40 50

Array becomes sorted.

Observation

Pass	Array State
Initial	50 20 40 10 30
Pass 1	20 40 10 30 50
Pass 2	20 10 30 40 50
Pass 3	10 20 30 40 50

Although only **3 passes** were required in this example, Bubble Sort can require **$n - 1$ passes** in the worst case.

Number of Passes in Bubble Sort

Suppose the array contains **n** elements.

Maximum number of passes required:

$n-1$

Example

Number of Elements	Maximum Passes
5	4
8	7
10	9
20	19

Number of Comparisons in Bubble Sort

During each pass, the number of comparisons decreases.

Pass Comparisons

1 $n - 1$

2 $n - 2$

3 $n - 3$

... ...

Last 1

Total Comparisons

$$(n - 1) + (n - 2) + \dots + 1$$
$$= \frac{n(n - 1)}{2}$$

Therefore,

Time Complexity

$O(n^2)$

Number of Passes in Selection Sort

Selection Sort repeatedly selects the smallest element and places it in its correct position.

Maximum Passes

$n-1$

Example

Elements	Maximum Passes
5	4
8	7
10	9

Number of Passes in Insertion Sort

Insertion Sort inserts one element into its correct position during each pass.

Maximum Passes

$n-1$

Example

Elements	Maximum Passes
5	4
8	7
10	9

Number of Passes in Merge Sort

Merge Sort divides the array into smaller parts and then merges them.

The number of merge levels (passes) is approximately

$$\log_2 n$$

Example

Elements	Merge Levels
8	3
16	4
32	5

Number of Passes in Quick Sort

Quick Sort partitions the array around a pivot.

The number of partitioning levels depends on the pivot selection.

Average Case

$O(\log n)$

Worst Case

$O(n)$

Comparison of Number of Passes

Sorting Algorithm	Maximum Passes
Bubble Sort	$n - 1$
Selection Sort	$n - 1$
Insertion Sort	$n - 1$
Merge Sort	$\log_2 n$ Levels
Quick Sort	Depends on Pivot

Python Demonstration (Bubble Sort Passes)

```
numbers = [50, 20, 40, 10, 30]
```

```
n = len(numbers)
```

```
for i in range(n - 1):
```

```
    print(f"\nPass {i + 1}")
```

```
    for j in range(n - i - 1):
```

```
        if numbers[j] > numbers[j + 1]:
```

```
            numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
```

```
    print(numbers)
```

Output

Pass 1

[20, 40, 10, 30, 50]

Pass 2

[20, 10, 30, 40, 50]

Pass 3

[10, 20, 30, 40, 50]

Pass 4

[10, 20, 30, 40, 50]

Notice that after Pass 3 the array is already sorted, but the standard Bubble Sort still performs the remaining pass unless an optimization (swap flag) is used.

Importance of Number of Passes

The number of passes helps us:

- Estimate the execution time.
 - Compare sorting algorithms.
 - Analyze algorithm efficiency.
 - Optimize sorting implementations.
 - Understand algorithm behavior.
-

Advantages

- Helps in complexity analysis.
 - Indicates algorithm progress.
 - Useful for comparing different sorting methods.
 - Assists in performance optimization.
-

Limitations

- Number of passes alone does not determine overall efficiency.
- Some algorithms perform fewer passes but more comparisons.
- Performance also depends on the number of swaps and memory usage.

2.8.5 Internal and External Sorting

Learning Objectives

After studying this section, students will be able to:

- Define Internal Sorting and External Sorting.
 - Differentiate between Internal and External Sorting.
 - Explain the characteristics of both sorting techniques.
 - Identify the advantages and limitations of each method.
 - Select an appropriate sorting technique based on data size.
 - Understand the applications of Internal and External Sorting.
-

2.8.5.1 Introduction

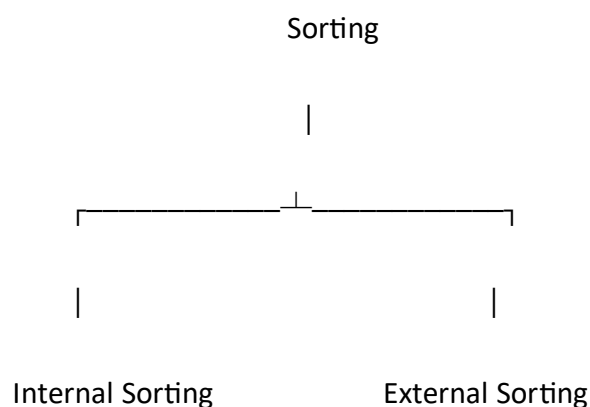
Sorting algorithms are classified based on **where the data is stored during the sorting process**.

If the entire dataset can fit into the **main memory (RAM)**, the sorting is called **Internal Sorting**.

If the dataset is too large to fit into RAM and must be stored on **secondary storage devices** such as hard disks or SSDs, the sorting is called **External Sorting**.

This classification is important because the performance of sorting algorithms depends on the storage location of the data.

Classification of Sorting



(RAM Based)

(Secondary Storage)

1. Internal Sorting

Definition

Internal Sorting is a sorting technique in which the entire dataset to be sorted is loaded into the main memory (RAM) before the sorting process begins.

Since all data is available in memory, sorting operations are performed quickly.

Characteristics of Internal Sorting

- Entire dataset fits into RAM.
- Faster execution.
- Low disk access.
- Suitable for small and medium-sized datasets.
- Most commonly used in programming applications.

Working of Internal Sorting

Input Data

|

Load Entire Data into RAM

|

Apply Sorting Algorithm

|

Sorted Data

Common Internal Sorting Algorithms

The following algorithms are examples of Internal Sorting:

- Bubble Sort
 - Selection Sort
 - Insertion Sort
 - Merge Sort
 - Quick Sort
 - Heap Sort
-

Advantages of Internal Sorting

- Fast execution.
 - Easy to implement.
 - Less input/output overhead.
 - Efficient for moderate-sized datasets.
 - Suitable for most applications.
-

Limitations of Internal Sorting

- Limited by RAM capacity.
 - Cannot efficiently process very large datasets.
 - Performance decreases if memory is insufficient.
-

Example

Suppose a computer has **8 GB RAM**, and the data to be sorted occupies **200 MB**.

Since the entire dataset easily fits into memory, the sorting can be performed using **Internal Sorting**.

2. External Sorting

Definition

External Sorting is a sorting technique used when the dataset is too large to fit into the main memory (RAM).

Only a small portion of the data is loaded into memory at a time, sorted, and then written back to secondary storage.

Characteristics of External Sorting

- Dataset is larger than RAM.
 - Uses secondary storage.
 - Data is processed in smaller chunks.
 - Requires multiple read/write operations.
 - Slower than Internal Sorting due to disk access.
-

Working of External Sorting

Large Data File

|

Divide into Small Chunks

|

Load One Chunk into RAM

|

Sort the Chunk

|

Store Sorted Chunk

|

Merge All Sorted Chunks

|

Final Sorted File

Common External Sorting Algorithms

Some commonly used External Sorting techniques are:

- External Merge Sort
- Polyphase Merge Sort
- Balanced Multi-way Merge Sort
- K-way Merge Sort

(Among these, External Merge Sort is the most widely used.)

Advantages of External Sorting

- Can process extremely large datasets.
 - Not limited by RAM size.
 - Suitable for database systems.
 - Efficient for file processing.
 - Used in big data applications.
-

Limitations of External Sorting

- Slower due to disk access.
 - Requires additional storage space.
 - More complex implementation.
 - High input/output operations.
-

Comparison Between Internal and External Sorting

Feature	Internal Sorting	External Sorting
Data Size	Small or Medium	Very Large
Storage Used	Main Memory (RAM)	Secondary Storage (Disk/SSD)
Speed	Faster	Slower
Memory Requirement	Entire data fits into RAM	Only part of the data fits into RAM
Disk Access	Minimal	Frequent
Complexity	Simple	More Complex
Cost	Lower	Higher
Examples	Bubble, Insertion, Selection, Merge, Quick	External Merge Sort, Polyphase Merge Sort

Internal vs External Sorting (Quick Comparison)

Internal Sorting

- ✓ Entire data fits in RAM
- ✓ Faster
- ✓ Less Disk Access
- ✓ Small Datasets

External Sorting

- ✓ Data larger than RAM

✓ Uses Hard Disk / SSD

✓ Slower

✓ Large Datasets

Applications of Internal Sorting

Internal Sorting is used in:

- Student record management systems.
 - Banking applications.
 - Library management systems.
 - Inventory management.
 - Mobile applications.
 - Desktop software.
 - Operating systems.
 - Compiler design.
-

Applications of External Sorting

External Sorting is used in:

- Database Management Systems (DBMS).
 - Big Data processing.
 - Search engines.
 - Cloud computing.
 - Data warehouses.
 - Scientific computing.
 - Log file processing.
 - Large-scale data analytics.
-

Python Demonstration (Internal Sorting)

```
numbers = [45, 12, 67, 23, 89, 10]
```

```
numbers.sort()
```

```
print("Sorted List:", numbers)
```

Output

Sorted List: [10, 12, 23, 45, 67, 89]

Python's built-in sort() method performs **Internal Sorting** because the entire list is loaded into memory.

Complexity Considerations

Type	Main Cost
Internal Sorting	CPU Time
External Sorting	Disk Input/Output (I/O) Time

In External Sorting, disk access is much slower than memory access, making I/O operations the dominant factor.

2.9 Bubble Sort

Learning Objectives

After studying this section, students will be able to:

- Define Bubble Sort.
- Explain the working principle of Bubble Sort.
- Write the Bubble Sort algorithm.
- Trace Bubble Sort using pass-by-pass execution.

- Implement Bubble Sort in Python.
 - Analyze the time and space complexity.
 - Explain the advantages, disadvantages, and applications of Bubble Sort.
-

2.9.1 Introduction

Bubble Sort is one of the simplest comparison-based sorting algorithms. It repeatedly compares **adjacent elements** in the list and swaps them if they are in the wrong order.

After every pass, the **largest element "bubbles up" to its correct position at the end of the array**. This process continues until the entire array is sorted.

Bubble Sort is easy to understand and implement, making it suitable for beginners learning sorting algorithms.

Definition

Bubble Sort is a comparison-based sorting algorithm that repeatedly compares adjacent elements and swaps them if they are in the wrong order until the entire list is sorted.

Key Characteristics

- Comparison-based sorting algorithm.
 - Stable sorting algorithm.
 - In-place sorting algorithm.
 - Simple implementation.
 - Suitable for small datasets.
 - Inefficient for large datasets.
-

How Bubble Sort Works

Bubble Sort compares two adjacent elements.

- If the left element is **greater** than the right element, they are swapped.

- Otherwise, no swap is performed.

After one complete pass:

- The **largest element reaches its correct position**.

The process repeats for the remaining unsorted portion of the array.

Working Principle

Start



Compare Adjacent Elements



Wrong Order?



Yes → Swap



No → Continue



End of Pass?



No → Compare Next Pair



Yes



Largest Element Fixed



Repeat Remaining Passes



Array Sorted

↓

Stop

Example

Consider the following unsorted array.

50 20 40 10 30

We will sort the array in **ascending order**.

Pass 1

Compare 50 and 20

50 20 40 10 30

↓

20 50 40 10 30

Swap performed.

Compare 50 and 40

20 50 40 10 30

↓

20 40 50 10 30

Swap performed.

Compare 50 and 10

20 40 50 10 30

↓

20 40 10 50 30

Swap performed.

Compare 50 and 30

20 40 10 50 30

↓

20 40 10 30 50

Swap performed.

After Pass 1

20 40 10 30 50

Largest element **50** is now in its correct position.

Pass 2

Compare 20 and 40

20 40 10 30 50

No swap.

Compare 40 and 10

20 40 10 30 50

↓

20 10 40 30 50

Swap.

Compare 40 and 30

20 10 40 30 50

↓

20 10 30 40 50

Swap.

After Pass 2

20 10 30 40 50

Element **40** reaches its correct position.

Pass 3

Compare 20 and 10

20 10 30 40 50

↓

10 20 30 40 50

Swap.

Compare remaining elements.

No swaps required.

After Pass 3

10 20 30 40 50

The array is now sorted.

Pass 4

No swaps are required.

The algorithm terminates.

Pass-wise Summary

Pass	Array After Pass
Initial	50 20 40 10 30
Pass 1	20 40 10 30 50

Pass 2	20 10 30 40 50
Pass 3	10 20 30 40 50
Pass 4	10 20 30 40 50

Bubble Sort Algorithm

Algorithm BubbleSort(A, n)

Input

A → Array

n → Number of elements

Step 1

Repeat for i = 0 to n-2

Step 2

Repeat for j = 0 to n-i-2

Step 3

If A[j] > A[j+1]

Swap A[j] and A[j+1]

Step 4

Repeat until all passes complete.

Step 5

Stop.

Python Program

```
def bubble_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n - 1):
```

```
        for j in range(n - i - 1):
```

```
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr

numbers = [50, 20, 40, 10, 30]

print("Original Array:", numbers)

bubble_sort(numbers)

print("Sorted Array :", numbers)
```

Output

Original Array: [50, 20, 40, 10, 30]

Sorted Array : [10, 20, 30, 40, 50]

Dry Run Table

Pass	Comparisons	Swaps	Array
Initial	—	—	50 20 40 10 30
1	4	4	20 40 10 30 50
2	3	2	20 10 30 40 50
3	2	1	10 20 30 40 50
4	1	0	10 20 30 40 50

Space Complexity

Bubble Sort uses only one temporary variable for swapping.

$O(1)$

Complexity Summary

Case	Time Complexity
Best	$O(n)$ (<i>Optimized Bubble Sort</i>)
Average	$O(n^2)$
Worst	$O(n^2)$
Space	$O(1)$

Advantages

- Easy to understand and implement.
 - Stable sorting algorithm.
 - In-place sorting (no extra memory required).
 - Suitable for small datasets.
 - Good for educational purposes.
-

Limitations

- Inefficient for large datasets.
 - Large number of comparisons.
 - Large number of swaps.
 - Poor performance compared to Merge Sort and Quick Sort.
-

Applications

Bubble Sort is used in:

- Educational purposes.
- Small datasets.
- Nearly sorted data (optimized version).
- Simple embedded systems.
- Demonstrating sorting concepts.

Bubble Sort Characteristics

Property	Value
Sorting Type	Comparison-Based
Stable	Yes
In-place	Yes
Adaptive	Yes (<i>Optimized Version</i>)
Best Case	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space Complexity	$O(1)$

2.9.2 Optimized Bubble Sort

2.9.2.1 Introduction

In the previous section, we studied the standard Bubble Sort algorithm.

One drawback of the standard Bubble Sort is that it performs **all $(n - 1)$ passes**, even if the array becomes sorted before completing all passes.

This results in unnecessary comparisons and increases execution time.

To overcome this limitation, an **Optimized Bubble Sort** is used.

The optimized version introduces a **Swap Flag** to determine whether any swapping occurred during a pass.

If no swaps occur during a complete pass, it means the array is already sorted, and the algorithm terminates immediately.

Need for Optimization

Consider the following sorted array.

10 20 30 40 50

The standard Bubble Sort still performs:

- Pass 1
- Pass 2
- Pass 3
- Pass 4

Although the array is already sorted.

These unnecessary passes waste CPU time.

The optimized algorithm avoids this problem.

Idea Behind Optimized Bubble Sort

During every pass,

- Initialize a variable named **swapped** as **False**.
- Whenever two elements are exchanged, set

swapped = True

After completing one pass,

If

swapped == False

No elements were exchanged.

Hence,

The array is already sorted.

Stop the algorithm immediately.

Example

Initial Array

10 20 30 40 50

Pass 1

Compare

10 and 20

No Swap

Compare

20 and 30

No Swap

Compare

30 and 40

No Swap

Compare

40 and 50

No Swap

Since

swapped = False

The algorithm terminates after the **first pass**.

Comparison with Standard Bubble Sort

Standard Bubble Sort	Optimized Bubble Sort
Performs all passes	Stops early if sorted
More comparisons	Fewer comparisons
Slower	Faster
Simple implementation	Slightly improved implementation

Algorithm

Algorithm OptimizedBubbleSort(A, n)

Input

A → Array

n → Number of Elements

Step 1

For i = 0 to n-2

 swapped = False

Step 2

For j = 0 to n-i-2

 If A[j] > A[j+1]

 Swap A[j] and A[j+1]

 swapped = True

Step 3

If swapped == False

 Stop

Step 4

Return Sorted Array

Step 5

Stop

Python Program

```
def optimized_bubble_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n - 1):
```

```
        swapped = False
```

```
        for j in range(n - i - 1):
```

```
    if arr[j] > arr[j + 1]:
        arr[j], arr[j + 1] = arr[j + 1], arr[j]
        swapped = True
    if not swapped:
        break
    return arr
```

```
numbers = [10, 20, 30, 40, 50]
print("Original Array :", numbers)
optimized_bubble_sort(numbers)
print("Sorted Array  :", numbers)
```

Output

Original Array : [10, 20, 30, 40, 50]

Sorted Array : [10, 20, 30, 40, 50]

Complexity Summary

Case	Time Complexity
Best	$O(n)$
Average	$O(n^2)$
Worst	$O(n^2)$
Space	$O(1)$

Advantages

- Eliminates unnecessary passes.
- Faster for already sorted or nearly sorted arrays.

- Simple modification to standard Bubble Sort.
 - Reduces execution time in the best case.
 - Maintains stability and in-place sorting.
-

Limitations

- Still inefficient for large datasets.
 - Average and worst-case complexities remain **$O(n^2)$** .
 - Performs more comparisons than advanced sorting algorithms such as Merge Sort and Quick Sort.
-

Comparison: Standard vs Optimized Bubble Sort

Feature	Standard Bubble Sort	Optimized Bubble Sort
Swap Flag	No	Yes
Early Termination	No	Yes
Best Case	$O(n^2)$	$O(n)$
Average Case	$O(n^2)$	$O(n^2)$
Worst Case	$O(n^2)$	$O(n^2)$
Space Complexity	$O(1)$	$O(1)$
Stable	Yes	Yes
In-place	Yes	Yes

2.10 Insertion Sort

(As per SPPU 2024 Pattern – Unit II)

Learning Objectives

After studying this section, students will be able to:

- Define Insertion Sort.
 - Explain the working principle of Insertion Sort.
 - Write the algorithm for Insertion Sort.
 - Trace the sorting process using pass-wise execution.
 - Implement Insertion Sort in Python.
 - Analyze the time and space complexity.
 - Explain the advantages, disadvantages, and applications of Insertion Sort.
-

2.10.1 Introduction

Insertion Sort is a simple comparison-based sorting algorithm that builds the sorted list **one element at a time**.

It works in a way similar to arranging playing cards in our hands. Initially, the first element is considered sorted. Then, each subsequent element is picked and inserted into its correct position in the already sorted part of the array.

Unlike Bubble Sort, which repeatedly swaps adjacent elements, Insertion Sort shifts larger elements one position to the right and inserts the current element (called the **key**) into its correct position.

Insertion Sort is efficient for **small datasets** and **nearly sorted arrays**.

Definition

Insertion Sort is a comparison-based sorting algorithm that builds a sorted array one element at a time by repeatedly inserting each new element into its correct position in the already sorted portion of the array.

Key Characteristics

- Comparison-based sorting algorithm.
- Stable sorting algorithm.
- In-place sorting algorithm.

- Adaptive algorithm (performs well for nearly sorted data).
 - Efficient for small datasets.
 - Simple to implement.
-

Working Principle

Initially,

- The **first element** is considered sorted.
 - The **second element** is compared with the first and inserted into the correct position.
 - The **third element** is inserted into the sorted portion.
 - This process continues until all elements are inserted into their correct positions.
-

Working Steps

1. Assume the first element is already sorted.
 2. Select the next element (called the **key**).
 3. Compare the key with elements on its left.
 4. Shift larger elements one position to the right.
 5. Insert the key into its correct position.
 6. Repeat the process for all remaining elements.
-

Working Representation

Unsorted Array



Assume First Element Sorted



Select Next Element (Key)



Compare with Left Side Elements



Shift Larger Elements Right



Insert Key at Correct Position



Repeat Until All Elements Are Sorted



Sorted Array

Example

Sort the following array in **ascending order**.

50 20 40 10 30

Pass 1

Sorted Part

50

Key = **20**

Compare

$20 < 50$

Shift 50.

Insert 20.

Result

20 50 40 10 30

Pass 2

Sorted Part

20 50

Key = **40**

Compare

$40 < 50$

Shift 50.

Compare

$40 > 20$

Insert after 20.

Result

20 40 50 10 30

Pass 3

Sorted Part

20 40 50

Key = **10**

Shift

50

Shift

40

Shift

20

Insert 10.

Result

10 20 40 50 30

Pass 4

Sorted Part

10 20 40 50

Key = **30**

Shift

50

Shift

40

Insert after 20.

Result

10 20 30 40 50

Pass-wise Summary

Pass	Key	Array After Pass
------	-----	------------------

Initial	—	50 20 40 10 30
---------	---	----------------

1	20	20 50 40 10 30
---	----	----------------

2	40	20 40 50 10 30
---	----	----------------

3	10	10 20 40 50 30
---	----	----------------

4	30	10 20 30 40 50
---	----	----------------

Algorithm

Algorithm InsertionSort(A, n)

Input

A → Array

n → Number of elements

Step 1

Assume first element is sorted.

Step 2

For i = 1 to n-1

key = A[i]

j = i-1

Step 3

While $j \geq 0$ and $A[j] > \text{key}$

Shift A[j] to A[j+1]

j = j-1

Step 4

Insert key at A[j+1]

Step 5

Repeat until all elements are processed.

Step 6

Stop.

Python Program

```
def insertion_sort(arr):
```

```
n = len(arr)
```

```
for i in range(1, n):
```

```
    key = arr[i]
```

```
    j = i - 1
```

```
    while j >= 0 and arr[j] > key:
```

```
        arr[j + 1] = arr[j]
```

```
        j -= 1
```

```
    arr[j + 1] = key
```

```
return arr
```

```
numbers = [50, 20, 40, 10, 30]
```

```
print("Original Array :", numbers)
```

```
insertion_sort(numbers)
```

```
print("Sorted Array :", numbers)
```

Output

Original Array : [50, 20, 40, 10, 30]

Sorted Array : [10, 20, 30, 40, 50]

2.11 Selection Sort

2.11.1 Introduction

Selection Sort is a simple comparison-based sorting algorithm that repeatedly selects the **smallest element** from the unsorted portion of the array and places it at its correct position in the sorted portion.

Unlike Bubble Sort, which performs many swaps, Selection Sort performs **only one swap in each pass**.

Initially, the first position is considered for placing the smallest element. After the first pass, the second position is considered, and the process continues until the array is completely sorted.

Selection Sort is easy to understand but is generally inefficient for large datasets because it performs many comparisons.

Definition

Selection Sort is a comparison-based sorting algorithm that repeatedly selects the smallest element from the unsorted part of the array and places it in its correct position.

Key Characteristics

- Comparison-based sorting algorithm.
 - In-place sorting algorithm.
 - Performs minimum number of swaps.
 - Not an adaptive algorithm.
 - Not a stable sorting algorithm (standard implementation).
 - Suitable for small datasets.
-

Working Principle

Selection Sort divides the array into two parts:

- **Sorted Part** (left side)
- **Unsorted Part** (right side)

During each pass:

1. Find the smallest element in the unsorted portion.
2. Swap it with the first element of the unsorted portion.
3. Increase the size of the sorted portion by one.
4. Repeat until all elements are sorted.

Example

Sort the following array in ascending order.

50 20 40 10 30

Pass 1

Find the smallest element.

50 20 40 10 30

Smallest = **10**

Swap 50 and 10.

10 20 40 50 30

Pass 2

Unsorted portion

20 40 50 30

Smallest = **20**

Already at correct position.

No swap required.

10 20 40 50 30

Pass 3

Unsorted portion

40 50 30

Smallest = **30**

Swap 40 and 30.

10 20 30 50 40

Pass 4

Unsorted portion

50 40

Smallest = **40**

Swap 50 and 40.

10 20 30 40 50

Array becomes sorted.

Pass-wise Summary

Pass	Smallest Element	Array After Pass
Initial	—	50 20 40 10 30
1	10	10 20 40 50 30
2	20	10 20 40 50 30
3	30	10 20 30 50 40
4	40	10 20 30 40 50

Algorithm

Algorithm SelectionSort(A, n)

Input

A → Array

n → Number of Elements

Step 1

For i = 0 to n-2

 min = i

Step 2

For j = i+1 to n-1

 If A[j] < A[min]

 min = j

Step 3

Swap A[i] and A[min]

Step 4

Repeat until all elements are sorted.

Step 5

Stop.

Python Program

```
def selection_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n - 1):
```

```
        min_index = i
```

```
        for j in range(i + 1, n):
```

```
            if arr[j] < arr[min_index]:
```

```
                min_index = j
```

```
    arr[i], arr[min_index] = arr[min_index], arr[i]
```

```
    return arr

numbers = [50, 20, 40, 10, 30]

print("Original Array :", numbers)

selection_sort(numbers)

print("Sorted Array  :", numbers)
```

Output

Original Array : [50, 20, 40, 10, 30]

Sorted Array : [10, 20, 30, 40, 50]

Dry Run Table

Pass	Minimum Element	Comparisons	Swaps	Array
Initial	—	—	—	50 20 40 10 30
1	10	4	1	10 20 40 50 30
2	20	3	0	10 20 40 50 30
3	30	2	1	10 20 30 50 40
4	40	1	1	10 20 30 40 50

Complexity Summary

Case	Time Complexity
Best	$O(n^2)$
Average	$O(n^2)$
Worst	$O(n^2)$
Space	$O(1)$

Characteristics of Selection Sort

Property	Value
Sorting Type	Comparison-Based
Stable	No (Standard Version)
In-place	Yes
Adaptive	No
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space Complexity	$O(1)$

Advantages

- Easy to understand and implement.
 - Performs fewer swaps than Bubble Sort.
 - Requires only constant extra memory.
 - Suitable for small datasets.
 - Simple algorithm for educational purposes.
-

Limitations

- Performs the same number of comparisons regardless of input order.
 - Inefficient for large datasets.
 - Not stable in its standard implementation.
 - Not adaptive for nearly sorted arrays.
-

Comparison: Bubble Sort vs Insertion Sort vs Selection Sort

Feature	Bubble Sort	Insertion Sort	Selection Sort
Method	Adjacent Swapping	Insert Key	Select Minimum
Stable	Yes	Yes	No
Adaptive	Yes (Optimized)	Yes	No
Best Case	$O(n)$	$O(n)$	$O(n^2)$
Average Case	$O(n^2)$	$O(n^2)$	$O(n^2)$
Worst Case	$O(n^2)$	$O(n^2)$	$O(n^2)$
Swaps	Many	Few	Minimum
Space	$O(1)$	$O(1)$	$O(1)$

Applications

Selection Sort is used in:

- Small datasets.
- Systems where memory is limited.
- Situations where minimizing the number of swaps is important.
- Educational demonstrations of sorting algorithms.

2.12 Quick Sort

2.12.1 Introduction

Quick Sort is one of the fastest and most widely used sorting algorithms. It follows the **Divide and Conquer** strategy to sort an array efficiently.

Unlike Bubble Sort or Selection Sort, Quick Sort does not repeatedly compare adjacent elements. Instead, it selects one element as a **Pivot**, partitions the array around the pivot, and recursively sorts the left and right subarrays.

Because of its excellent average-case performance, Quick Sort is commonly used in system software, programming libraries, and high-performance applications.

Definition

Quick Sort is a Divide and Conquer sorting algorithm that selects a pivot element, partitions the array into two parts, and recursively sorts the subarrays until the entire array becomes sorted.

Key Characteristics

- Comparison-based sorting algorithm.
 - Divide and Conquer technique.
 - Recursive algorithm.
 - In-place sorting algorithm.
 - Generally **not stable**.
 - Very efficient for large datasets.
-

Principle of Divide and Conquer

Quick Sort works in three major steps:

1. Divide

Choose one element as the **Pivot** and partition the array into two parts.

- Elements smaller than Pivot → Left side
 - Elements larger than Pivot → Right side
-

2. Conquer

Recursively apply Quick Sort on

- Left Subarray
 - Right Subarray
-

3. Combine

Unlike Merge Sort, **no explicit merging is required**.

Once recursive calls complete, the array is already sorted.

Pivot Element

A **Pivot** is the element around which the array is partitioned.

Choosing a good pivot improves the efficiency of Quick Sort.

Pivot Selection Methods

There are several ways to choose the pivot.

1. First Element

Example

40 20 50 10 30

Pivot = 40

Simple to implement but may produce poor performance for already sorted arrays.

2. Last Element

Example

40 20 50 10 30

Pivot = 30

This is one of the most commonly used methods and is used in many textbook implementations.

3. Middle Element

Example

40 20 50 10 30

Pivot = 50

Reduces the chance of worst-case behavior for some input distributions.

4. Random Pivot

A random element is selected as the pivot.

This reduces the probability of worst-case performance.

5. Median of Three

Choose

- First element
- Middle element
- Last element

The median (middle value) among these three is selected as the pivot.

This often produces balanced partitions and improves performance.

Example

Array

50 20 40 10 30

Choose the **last element** as Pivot.

Pivot = 30

Partitioning produces

20 10 30 50 40

Now,

Left Subarray

20 10

Right Subarray

50 40

These two subarrays are sorted recursively.

Final Result

10 20 30 40 50

Why is Pivot Important?

The efficiency of Quick Sort depends heavily on the pivot.

Good Pivot

Produces nearly equal partitions.

Example

100 Elements

↓

50 Elements | Pivot | 49 Elements

Performance becomes

$O(n \log n)$

Poor Pivot

Produces very uneven partitions.

Example

99 Elements | Pivot | 0 Elements

Performance becomes

$O(n^2)$

Advantages of Good Pivot Selection

- Reduces recursion depth.
- Improves execution speed.
- Produces balanced partitions.
- Achieves average-case efficiency.

2.12.2 Partitioning Techniques in Quick Sort

2.12.2.1 Introduction

The **partitioning process** is the most important step in the Quick Sort algorithm.

Partitioning rearranges the elements of an array around a selected **Pivot** such that:

- All elements smaller than the pivot are placed on the left side.
- The pivot reaches its correct position.
- All elements greater than the pivot are placed on the right side.

After partitioning, Quick Sort recursively sorts the left and right subarrays.

Definition

Partitioning is the process of rearranging the elements of an array around a pivot element so that smaller elements are placed before the pivot and larger elements are placed after it.

Need for Partitioning

Partitioning helps Quick Sort by:

- Dividing a large problem into smaller subproblems.
- Placing the pivot in its final sorted position.
- Reducing unnecessary comparisons.
- Improving sorting efficiency.

Types of Partitioning Techniques

The two most commonly used partitioning schemes are:

1. **Lomuto Partition Scheme**
 2. **Hoare Partition Scheme**
-

1. Lomuto Partition Scheme

Introduction

The **Lomuto Partition Scheme** is one of the simplest methods used in Quick Sort.

Characteristics:

- Uses the **last element** as the pivot.
 - Maintains one pointer (i) for the boundary of smaller elements.
 - Scans the array using another pointer (j).
 - Places the pivot in its correct position after scanning.
-

Algorithm Steps

Suppose

A = [50, 20, 40, 10, 30]

Pivot = Last Element

30

Initialize

i = low - 1

Now scan from left to right.

Step 1

Compare

$50 < 30$?

No

No swap.

Step 2

Compare

$20 < 30$?

Yes

Increase i

Swap

$50 \leftrightarrow 20$

Array

20 50 40 10 30

Step 3

Compare

$40 < 30$?

No

Step 4

Compare

$10 < 30$?

Yes

Increase i

Swap

50 $\leftrightarrow$ 10

Array

20 10 40 50 30

Final Step

Swap Pivot with

A[i+1]

Swap

40 $\leftrightarrow$ 30

Final Partition

20 10 30 50 40

Pivot is now in its correct position.

Lomuto Partition Algorithm

Algorithm LomutoPartition(A, low, high)

pivot = A[high]

i = low - 1

For j = low to high - 1

 If A[j] < pivot

 i = i + 1

 Swap A[i] and A[j]

Swap A[i+1] and A[high]

Return i+1

Python Program (Lomuto)

```
def partition(arr, low, high):  
    pivot = arr[high]  
    i = low - 1  
    for j in range(low, high):  
        if arr[j] < pivot:  
  
            i += 1  
            arr[i], arr[j] = arr[j], arr[i]  
    arr[i + 1], arr[high] = arr[high], arr[i + 1]  
    return i + 1  
  
numbers = [50, 20, 40, 10, 30]  
index = partition(numbers, 0, len(numbers) - 1)  
print("Pivot Index :", index)  
print(numbers)
```

Output

Pivot Index : 2

[20, 10, 30, 50, 40]

Advantages of Lomuto Partition

- Very easy to understand.
 - Simple implementation.
 - Popular in textbooks.
 - Suitable for beginners.
-

Limitations

- Performs more swaps.
 - Less efficient than Hoare Partition.
 - Poor performance with repeated elements.
-

2. Hoare Partition Scheme

Introduction

The **Hoare Partition Scheme** was developed by **Sir Tony Hoare**, the inventor of Quick Sort.

Characteristics:

- Uses the **first element** as the pivot.
- Uses two pointers.

One pointer moves from

- Left → Right

The other pointer moves from

- Right → Left

When incorrect elements are found, they are swapped.

Working

Array

50 20 40 10 30

Pivot

50

Left Pointer

Starts from beginning.

Right Pointer

Starts from end.

Scan

Left pointer stops at

50

Right pointer stops at

30

Swap

$50 \leftrightarrow 30$

Array

30 20 40 10 50

Continue until pointers cross.

Partition completed.

Hoare Partition Algorithm

Algorithm HoarePartition(A, low, high)

pivot = A[low]

$i = \text{low} - 1$

$j = \text{high} + 1$

Repeat

 Repeat $i = i + 1$

 Until $A[i] \geq \text{pivot}$

 Repeat $j = j - 1$

 Until $A[j] \leq \text{pivot}$

 If $i \geq j$

 Return j

Swap A[i] and A[j]

Python Program (Hoare Partition)

```
def hoare_partition(arr, low, high):  
    pivot = arr[low]  
    i = low - 1  
    j = high + 1  
    while True:  
        i += 1  
        while arr[i] < pivot:  
            i += 1  
        j -= 1  
        while arr[j] > pivot:  
            j -= 1  
        if i >= j:  
            return j  
        arr[i], arr[j] = arr[j], arr[i]  
numbers = [50, 20, 40, 10, 30]  
index = hoare_partition(numbers, 0, len(numbers)-1)  
print(index)  
print(numbers)
```

Output

3

[30, 20, 40, 10, 50]

Comparison Between Lomuto and Hoare Partition

Feature	Lomuto Partition	Hoare Partition
Pivot Position	Last Element	First Element (commonly)
Number of Pointers	One Main Pointer + Scanner	Two Pointers
Swaps	More	Fewer
Performance	Slightly Slower	Faster
Efficiency	Lower	Higher
Ease of Implementation	Easier	Slightly More Complex
Suitable For	Beginners	Efficient Implementations

Complexity Analysis

Both partitioning schemes require scanning the array once.

Time Complexity

$O(n)$

Space Complexity

$O(1)$

Advantages of Partitioning

- Efficiently divides the array.
- Reduces problem size.
- Supports recursive sorting.
- Improves Quick Sort performance.

2.12.3 Quick Sort Algorithm, Complete Dry Run and Python Implementation

2.12.3 Quick Sort Algorithm

Quick Sort repeatedly performs the following operations:

1. Select a Pivot.
2. Partition the array around the Pivot.
3. Apply Quick Sort recursively to the left subarray.
4. Apply Quick Sort recursively to the right subarray.
5. Stop when the subarray contains zero or one element.

Complete Algorithm

Algorithm QuickSort(A, low, high)

Input

A → Array

low → Starting Index

high → Ending Index

Step 1

If low < high

Step 2

pivot = Partition(A, low, high)

Step 3

QuickSort(A, low, pivot-1)

Step 4

QuickSort(A, pivot+1, high)

Step 5

Stop

Example

Sort the following array.

50 20 40 10 30

Choose the **last element** as Pivot.

First Recursive Call

Array

50 20 40 10 30

Pivot

30

After Partition

20 10 30 50 40

Pivot Position = **2**

Subarrays

Left

20 10

Right

50 40

Second Recursive Call (Left)

Array

20 10

Pivot

10

After Partition

10 20

Left Subarray

Empty

Right Subarray

Empty

Left side completed.

Third Recursive Call (Right)

Array

50 40

Pivot

40

After Partition

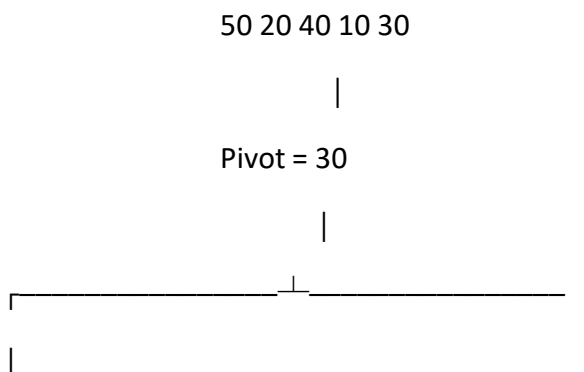
40 50

Right side completed.

Final Sorted Array

10 20 30 40 50

Recursion Tree



20 10

|

Pivot =10

|

10 20

50 40

|

Pivot =40

|

40 50

Python Program

```
def partition(arr, low, high):
```

```
    pivot = arr[high]
```

```
    i = low - 1
```

```
    for j in range(low, high):
```

```
        if arr[j] <= pivot:
```

```
            i += 1
```

```
            arr[i], arr[j] = arr[j], arr[i]
```

```
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
```

```
    return i + 1
```

```
def quick_sort(arr, low, high):
```

```
    if low < high:
```

```
        pivot = partition(arr, low, high)
```

```
        quick_sort(arr, low, pivot - 1)
```

```
        quick_sort(arr, pivot + 1, high)
```

```
numbers = [50, 20, 40, 10, 30]
```

```
print("Original Array :", numbers)
quick_sort(numbers, 0, len(numbers)-1)
print("Sorted Array :", numbers)
```

Output

Original Array : [50, 20, 40, 10, 30]

Sorted Array : [10, 20, 30, 40, 50]

Complexity Analysis

Suppose the array contains **n** elements.

Best Case

When the pivot divides the array into two equal halves.

$O(n \log n)$

Average Case

Balanced partitioning occurs most of the time.

$O(n \log n)$

Worst Case

Occurs when the smallest or largest element is repeatedly chosen as the pivot.

Example:

10 20 30 40 50

or

50 40 30 20 10

Time Complexity

$O(n^2)$

Space Complexity

Quick Sort requires stack memory due to recursive function calls.

Average Case

$O(\log n)$

Worst Case

$O(n)$

Complexity Summary

Case	Time Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$
Space (Average)	$O(\log n)$
Space (Worst)	$O(n)$

Characteristics of Quick Sort

Property	Value
Sorting Type	Comparison-Based
Technique	Divide and Conquer
Stable	No
In-place	Yes
Recursive	Yes
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$

Worst Case	$O(n^2)$
Space Complexity	$O(\log n)$ (Average)

Advantages

- Very fast in practice.
 - Efficient for large datasets.
 - Uses Divide and Conquer.
 - Requires little extra memory.
 - Widely used in standard libraries.
-

Limitations

- Worst-case complexity is $O(n^2)$.
 - Performance depends on pivot selection.
 - Recursive calls may increase stack usage.
 - Standard implementation is not stable.
-

Applications

Quick Sort is widely used in:

- Standard library sorting functions.
 - Database systems.
 - Compiler optimization.
 - Operating systems.
 - Large datasets stored in memory.
 - Searching and indexing applications.
-

Comparison with Previous Sorting Algorithms

Feature	Bubble	Insertion	Selection	Quick
Technique	Swapping	Insertion	Selection	Divide & Conquer
Stable	Yes	Yes	No	No
Best Case	$O(n)$	$O(n)$	$O(n^2)$	$O(n \log n)$
Average Case	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$
Worst Case	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$
Space	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$

2.13 Merge Sort

2.13.1 Introduction

Merge Sort is an efficient comparison-based sorting algorithm that follows the **Divide and Conquer** technique. It repeatedly divides an array into smaller subarrays until each subarray contains only one element. Then, these subarrays are merged in a sorted manner to produce the final sorted array.

Unlike Quick Sort, Merge Sort always divides the array into two nearly equal halves, resulting in a predictable time complexity of **$O(n \log n)$** in all cases.

Merge Sort is a **stable sorting algorithm**, making it suitable for applications where the relative order of equal elements must be preserved.

Definition

Merge Sort is a Divide and Conquer sorting algorithm that recursively divides an array into two halves, sorts each half independently, and merges the sorted halves to produce the final sorted array.

Key Characteristics

- Comparison-based sorting algorithm.
- Uses Divide and Conquer strategy.
- Stable sorting algorithm.

- Recursive algorithm.
 - Not an in-place algorithm.
 - Suitable for large datasets.
-

Divide and Conquer Strategy

Merge Sort consists of three major steps.

Step 1: Divide

Divide the array into two equal halves.

Step 2: Conquer

Recursively divide each half until every subarray contains only one element.

Step 3: Merge

Merge the sorted subarrays into larger sorted arrays until the complete array is sorted.

Difference Between Quick Sort and Merge Sort

Quick Sort	Merge Sort
Partition first	Divide first
No merge operation	Merge operation required
Pivot selection required	No pivot required
Usually in-place	Requires extra memory
Not stable	Stable

Example

Sort the following array.

38 27 43 3 9 82 10

Step 1

Divide into two halves.

38 27 43 3

9 82 10

Step 2

Divide again.

38 27

43 3

9 82

10

Step 3

Divide until each subarray contains one element.

38

27

43

3

9

82

10

Step 4

Merge in sorted order.

27 38

3 43

9 82

10

Step 5

Merge again.

3 27 38 43

9 10 82

Step 6

Final Merge

3 9 10 27 38 43 82

Complete Working

38 27 43 3 9 82 10

↓

38 27 43 3 9 82 10

↓

38 27 43 3 9 82 10

↓

38 27 43 3 9 82 10

↓

27 38 3 43 9 82 10

↓

3 27 38 43

9 10 82

↓

3 9 10 27 38 43 82

Merge Operation

The merge operation combines two already sorted arrays into a single sorted array.

Example

Left

10 30 50

Right

20 40 60

Merge

10 20 30 40 50 60

The merge operation repeatedly compares the first elements of both subarrays and copies the smaller element into the result array.

Merge Algorithm

Algorithm Merge(A, left, mid, right)

Step 1

Create two temporary arrays

Left[]

Right[]

Step 2

Copy elements into temporary arrays

Step 3

Compare first elements of both arrays

Step 4

Copy the smaller element into the original array

Step 5

Repeat until one array becomes empty

Step 6

Copy remaining elements

Step 7

Stop

Merge Sort Algorithm

Algorithm MergeSort(A, left, right)

Step 1

If left < right

Step 2

mid = (left + right) // 2

Step 3

MergeSort(A, left, mid)

Step 4

MergeSort(A, mid+1, right)

Step 5

Merge(A, left, mid, right)

Step 6

Stop

Python Program

```

def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

```

```

numbers = [38, 27, 43, 3, 9, 82, 10]

```

```
print("Original Array :", numbers)

merge_sort(numbers)

print("Sorted Array  :", numbers)
```

Output

Original Array : [38, 27, 43, 3, 9, 82, 10]

Sorted Array : [3, 9, 10, 27, 38, 43, 82]

Dry Run Table

Step	Operation	Result
1	Divide	38 27 43 3 9 82 10
2	Divide	38 27 43 3 / 9 82 10
3	Divide	Single element arrays
4	Merge	27 38 / 3 43 / 9 82 /10
5	Merge	3 27 38 43 / 9 10 82
6	Merge	3 9 10 27 38 43 82

Complexity Summary

Case	Time Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$
Space	$O(n)$

Characteristics of Merge Sort

Property	Value
Sorting Type	Comparison-Based
Technique	Divide and Conquer
Stable	Yes
In-place	No
Recursive	Yes
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$
Space Complexity	$O(n)$

Advantages

- Stable sorting algorithm.
 - Guaranteed $O(n \log n)$ time complexity.
 - Suitable for large datasets.
 - Performs well for linked lists.
 - Efficient for external sorting where data cannot fit entirely in memory.
-

Limitations

- Requires additional memory for merging.
 - Recursive implementation increases stack usage.
 - Slower than Quick Sort for many in-memory applications due to extra memory operations.
-

Applications

Merge Sort is widely used in:

- External sorting of very large files.
- Sorting linked lists.
- Database systems.
- Parallel processing.
- Distributed computing.
- Stable sorting requirements.

Comparison with Other Sorting Algorithms

Feature	Bubble	Insertion	Selection	Quick	Merge
Stable	Yes	Yes	No	No	Yes
In-place	Yes	Yes	Yes	Yes	No
Best Case	$O(n)$	$O(n)$	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
Average Case	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
Worst Case	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$
Extra Space	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$	$O(n)$